

Final Report

“FURBOT”



FurBot is a web-based assistant that combines smart chatbot support with personalized pet profile management, helping dog owners easily handle everyday needs—from finding parks and grooming services to managing vet visits and reminders

WINTER 2025
CSIS 4495 - 001

PRESENTED BY:
CHARLIE MEDIALDEA – 300374504

Table of Contents

Introduction	4
I. Introducing the domain, and overview of the background, and context of the research	4
II. Problem Statement.....	4
III. Relevant Literature and Research.....	5
IV. Hypothesis, Assumptions, and Potential Benefit of Research	5
Summary of the Research Project	6
Changes to the proposal	7
Features Added:	7
Technology Stack.....	8
Project Completion Timeline	12
System Architecture (MVC Pattern)	12
Gantt Chart	13
Sprint 1 (January 18 – 25, 2025)	13
Sprint 2 (February 3 – 8, 2025)	14
Sprint 3 (February 9 – 13, 2025)	15
Sprint 4 (February 14 – 18, 2025)	15
Sprint 5 (February 19 – March 10, 2025)	16
Sprint 6 (March 11 – 18, 2025)	18
Sprint 7 (March 24 – 30, 2025)	18
Sprint 8 (March 29, - April 13, 2025)	19
Implemented Features	21
Implemented Feature 1: Login	21
Implemented Feature 2: Registration	23
Implemented Feature 3: Dashboard	25
Implemented Feature 4: Pet Profile Management.....	27
Implemented Feature 5: Pet Care Reminder and Scheduling	33
Implemented Feature 6: Chatbot.....	42
Implemented Feature 7: Saving to Favorites	48

Implemented Feature 8: Automated Email Reminder	52
Evaluation and Techniques	54
Reflections/Discussions	57
Work Date/Hours logs	58
Concluding Remarks.....	59
References	60
Appendix A: Installation Guide	62
Appendix B: User Guide.....	63
Getting Started.....	64
Access FurBot	64
Sign Up / Login	64
Managing Your Pet Profile	66
Add a Pet.....	66
Delete a Pet.....	67
Viewing Pet Profile and Setting a Reminders for a Pet	68
Adding a Reminder	68
Delete a Reminder	70
Email Notification	70
Using the Chatbot	71
Example commands:	71
Adding result to favorites and viewing.....	72
Need Help or Want to Suggest a Feature?	73
Appendix C: Survey Questionnaire	74

Introduction

I. Introducing the domain, and overview of the background, and context of the research

FurBot was inspired by a personal journey as a dog owner migrating to Canada and facing challenges in navigating an unfamiliar environment. Without local connections, finding dog-friendly facilities, reliable grooming services, and essential pet care resources was overwhelming. This experience revealed a significant gap in accessible, localized, and reliable information for pet owners, particularly in urban settings. It became evident that many dog owners, especially newcomers, share similar struggles in identifying pet-friendly options and resources.

Combining pet care and technology, FurBot is a web-based chatbot designed to provide pet owners with quick and reliable access to essential services and information. FurBot uses a straightforward approach by leveraging predefined intents and responses to handle user queries. Through natural language processing (NLP), the chatbot is trained to recognize user input and match it to the most relevant predefined answers, such as locating nearby veterinary clinics, grooming salons, or dog parks. Additionally, FurBot can assist users with process-related questions, such as guidelines for traveling with pets to the US. This approach ensures an efficient and user-friendly experience while consolidating resources into a single platform, helping pet owners save time and navigate pet care services with ease.

II. Problem Statement

Here are some questions that help frame the problem:

- How can a chatbot provide reliable and real-time assistance to dog owners in finding localized pet care resources?
- What solutions can effectively bridge the gap between dog owners and small pet-related businesses?

Dog owners often struggle to access reliable and localized information about pet-friendly facilities and services, such as veterinary clinics, grooming salons, and dog parks. This challenge is further compounded when navigating process-related tasks, like traveling with pets or understanding local pet care regulations. A centralized, chatbot-driven platform like FurBot can address these issues by leveraging predefined intents and responses, supported by natural language processing (NLP).

III. Relevant Literature and Research

During the pandemic, much of the focus shifted toward essential human healthcare services, such as online consultations, often overlooking the need for accessible and reliable pet care solutions. This gap in localized pet care services highlighted the challenges pet owners face in finding nearby veterinary clinics, grooming salons, and pet supply stores. FurBot addresses this gap by consolidating vital pet care information into a single, chatbot-driven platform. By leveraging predefined intents and natural language processing (NLP), FurBot provides a streamlined experience tailored to users' locations, simplifying access to reliable resources and helping pet owners save time while making informed decisions about their pets' needs.

Advancements in chatbot technology have demonstrated the potential to improve accessibility and convenience for pet owners. Platforms like Woof.AI offer personalized product recommendations to enhance user experiences (Woof.AI, 2024). Similarly, AI-powered veterinary chatbots support scheduling, answer frequently asked questions, and manage reminders for vaccinations, providing round-the-clock assistance (FastBots, 2024). Building on these innovations, FurBot offers a userfriendly and efficient platform that provides real-time assistance through predefined questions and answers, supported by NLP. It ensures users receive accurate, personalized, and location-specific recommendations, meeting a wide range of needs, including finding pet-friendly spaces, accessing essential services, and addressing process-related queries like traveling with pets.

IV. Hypothesis, Assumptions, and Potential Benefit of Research

Hypotheses and Assumptions

FurBot aims to address key challenges pet owners face in accessing reliable, localized pet care services. Localized recommendations will reduce search time for services like veterinary clinics, grooming salons, and dog parks, while personalized suggestions tailored to user profiles will enhance satisfaction. The interactive Q&A chatbot will improve engagement by providing real-time answers to common pet care questions, ensuring a seamless user experience. The development of FurBot assumes that pet owners prefer a centralized platform offering quick access to multiple pet-related services, as well as personalized and location-specific recommendations. It also assumes that real-time assistance via a chatbot is more practical and engaging than traditional search methods.

Expected Benefits

For users, FurBot simplifies pet care by acting as a one-stop platform, saving time and offering tailored recommendations. Real-time assistance enhances usability, making FurBot a dependable tool for pet owners. For the community, FurBot increases visibility for local pet service providers and improves access to timely resources, fostering better pet welfare and stronger connections within local communities.

Summary of the Research Project

FurBot is a web-based intelligent assistant designed to support dog owners in managing their pets' everyday needs through a seamless combination of chatbot functionality and pet profile management. The platform was developed to address the gap in centralized, personalized care tools for pet parents, offering a user-friendly interface backed by practical features like appointment reminders, grooming suggestions, and travel guidance.

In its final form, FurBot allows users to register, log in, and create detailed pet profiles. The system can schedule and send automated email reminders for important events such as vaccinations and grooming sessions. A key feature of the project is the AI-powered chatbot, which provides real-time assistance with common queries related to parks, dog-friendly establishments, travel requirements, grooming services, and supplies.

The backend was built with a focus on scalability and modular development, using MongoDB for data management and Flask for the API. The system supports dynamic chatbot intent expansion and integrates CRON-based scheduling for timely reminders. By centralizing key pet care functions and enhancing them with AI support, FurBot contributes to improving pet wellness, streamlining daily care routines, and supporting responsible pet ownership.

Changes to the proposal

Initially, the goal of the project was to develop a chatbot application for answering pet-related queries. However, based on the professor's guidance and iterative feedback from users, the scope of the project was expanded significantly. The following key changes were made:

Features Added:

1. User Authentication (JWT)

Originally, the chatbot did not require login functionality. However, to support personalization (e.g., saving pet profiles and reminders), authentication was added using JSON Web Tokens (JWT).

2. Pet Profile Management

Users can now create and manage detailed pet profiles, capturing pet-specific information such as name, breed, and birthdate. This core feature enhances the user experience by offering a more personalized and thoughtful approach to pet care.

3. Reminders System

A dedicated reminder system has been introduced, allowing users to schedule important pet care activities such as vaccinations, grooming, and vet visits. This feature helps ensure that critical health and wellness tasks are not overlooked, supporting responsible pet ownership and contributing to the overall well-being of pets.

4. Email Notification Integration

With the integration of Flask-Mail, users no longer need to log in to check their pet care reminders — personalized email notifications are automatically sent for upcoming appointments such as vaccinations, grooming, or vet visits. The system checks for scheduled reminders daily, ensuring that users receive timely updates straight to their inbox. This feature adds a convenient, hands-off layer to pet care, offering a more personal and proactive experience for pet owners.

5. Smart/Pro-active Notifications

Leveraging the existing reminder system and further investing in making FurBot smarter, a proactive notification feature has been implemented. The system intelligently checks the last recorded reminders for services like grooming or vaccinations, and if they exceed a defined threshold (e.g., 30 days for grooming and 365 days for vaccination), it automatically sends a personalized email to the user suggesting it's time to schedule a new appointment. This smart feature ensures pets receive timely care without

requiring manual input from the user, reflecting a more intuitive and thoughtful approach to pet wellness management.

6. Integration with Yelp Fusion API (External API Free Subscription)

The chatbot now fetches real-time data on pet-friendly businesses like groomers, vets, and parks request, enhancing response relevance.

7. Location-Aware NLP Queries

In response to user feedback requesting more specific, location-based results, a location-aware natural language processing feature has been implemented using spaCy. Due to time constraints and the need for a solution that was easy to implement, spaCy was chosen for its out-of-the-box support for entity recognition. The chatbot can now detect and extract city names from user queries (e.g., "Looking for dog parks in Richmond"), enabling it to deliver more accurate and relevant location-specific information. This enhancement makes FurBot more context-aware and improves the overall user experience.

8. Deployment to Production

Initially planned as a local-only application, the backend was successfully deployed to production using Railway, making it accessible online for testing and demo purposes.

Technology Stack

➤ **Frontend:**

- **Vue.js**

No change. Used to create a responsive and user-friendly interface.

➤ **Backend and Framework:**

- **Flask (Python)**

No change. Continued to provide a lightweight yet powerful backend framework.

- **Email Service: Flask-Mail**

Newly added to enable the system to send automated, personalized email reminders to users. Since the project is built using the Flask framework, Flask-Mail was a natural choice due to its seamless integration and ease of use. It allowed for quick setup of an email notification system without the need for complex configurations. This feature enhances user engagement by delivering timely updates directly to their inbox, supporting a more proactive and user-friendly experience.

- **Task Scheduler: APScheduler**

Newly added to automate the dispatch of reminder emails. APScheduler was integrated with Flask to schedule daily tasks that trigger email reminders through Flask-Mail. This combination ensures that reminders are sent consistently and on time without manual intervention, helping maintain a smooth and reliable user experience. By automating this process, the system becomes more efficient and scalable, especially as the number of users and reminders grows.

➤ **Authentication and Security:**

- **JWT (JSON Web Tokens)**

Newly added. Enable secure user login and access control. JWT was recommended by many developer forums and is widely adopted in modern web development. It offers a good balance between ease of implementation and scalability, making it suitable for both small projects and larger systems that involve APIs or multiple services.

➤ **Database:**

- **MongoDB Atlas**

No changes were made to the database choice. MongoDB Atlas continues to be used for storing user data, pet profiles, and reminders. While I have industry experience working with SQL (Structured Query Language), I chose MongoDB for this project to gain hands-on experience with NoSQL databases and explore document-oriented data modeling. This decision also aligned with the flexible and scalable nature of the application.

➤ **Natural Language Processing:**

- **String Matching: FuzzyWuzzy**

No changes were made in this area. FuzzyWuzzy continues to be used for fuzzy keyword matching to handle user typos and near-matches. This approach was inspired by my previous role, where I served as the subject matter expert (SME) for our application. During the early stages of chatbot development in that role, I worked closely with the development team, and we initially used string matching to define and recognize basic user intents. That experience influenced my decision to adopt a similar method for this project, especially given my foundational knowledge in natural language processing at the time. Through research, I found FuzzyWuzzy to be a simple, Python-based solution well-suited for entry-level implementations.

- **spaCy**

Newly added to enable the chatbot to extract location information from user queries, allowing it to deliver more accurate and context-aware

responses. This approach was chosen because spaCy offers out-of-the-box support for named entity recognition (NER) and is easy to use, making it an ideal choice for entry-level natural language processing tasks within the Python ecosystem.

➤ **External API and Integrations:**

- **Railway**

Newly added to deploy the Flask backend along with MongoDB integration into a live environment. Railway was chosen for its simplicity and ease of use, offering free tier and allowing smooth deployment of the full stack. This setup provided valuable hands-on experience with continuous integration and continuous deployment (CI/CD), helping to bridge development and production in a more automated and efficient way.

- **Yelp Fusion API**

Newly added to provide real-time data on nearby pet-related services, such as groomers, veterinarians, and pet stores, based on city names detected in user input. This integration enhances the chatbot's usefulness by offering location-specific and practical responses. Yelp Fusion was selected due to its extensive business database, ease of integration, and the availability of a free subscription tier for development and testing purposes—making it a cost-effective choice during for this project.

➤ **Development Tools:**

- **ChatGPT (OpenAI) – Paid subscription**

Used extensively as a conversational assistant throughout the project. ChatGPT supported backend and frontend development by offering help with coding, debugging, and logic design. It also contributed to shaping the chatbot's behavior and flow by providing guidance on conversational structure and natural language handling. Beyond development, ChatGPT was instrumental in enhancing the quality and clarity of project reports, offering suggestions for wording, structure, and presentation. Additionally, it played a key role in helping me understand and explore new technologies that I used for the first time in this project, such as Flask-Mail, APScheduler, JWT, Flask and MongoDB.

- **GitHub Copilot – AI Coding Assistant (Integrated with Visual Studio Code)**

Used as a built-in assistant within Visual Studio Code to support both backend and frontend development. GitHub Copilot provided real-time code suggestions, helping accelerate the coding process and improve efficiency. When used alongside ChatGPT, it offered a powerful combination — with Copilot suggesting code directly within the IDE, and ChatGPT providing deeper explanations, logic validation, and troubleshooting support. This collaboration significantly enhanced my development workflow and confidence in working with unfamiliar technologies.

- **JIRA – Free subscription**

Used for project tracking, task management, and logging hours worked throughout the development of the project. JIRA helped maintain clear visibility over progress, deadlines, and milestones, supporting a more organized and accountable workflow even as a solo developer.

- **GitHub**

Version control and project repository management.

➤ **Document Creation**

- **Canva (Free subscription)**

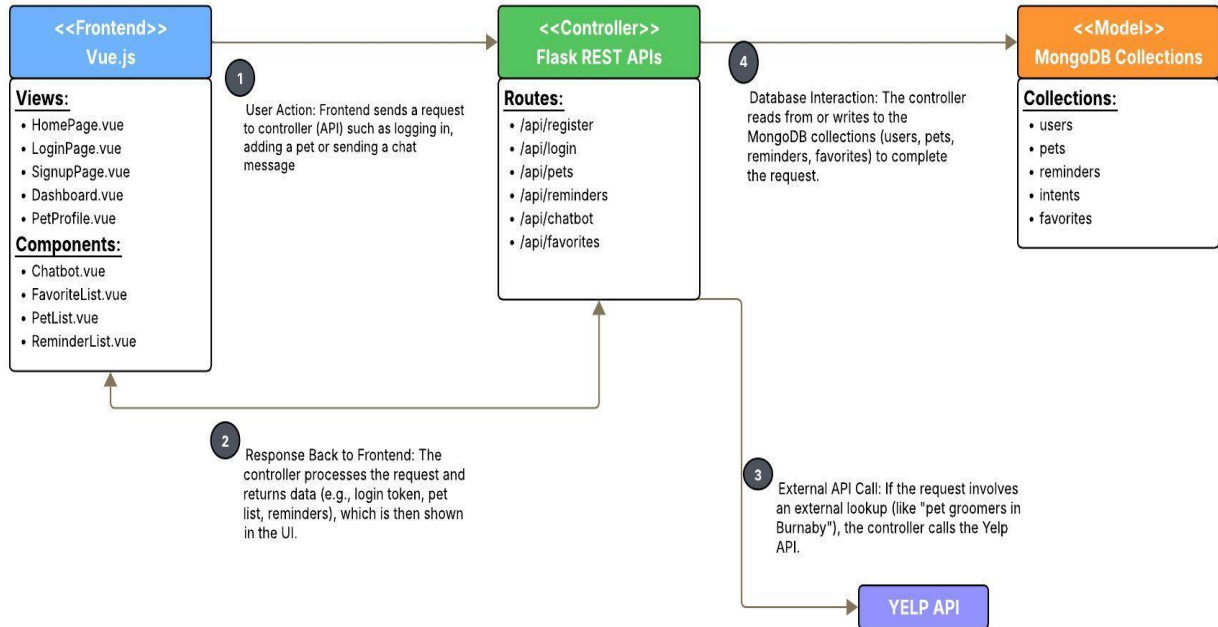
Canva was used to enhance the visual presentation of key project documents, including the proposal, progress report, and midterm report. It allowed for a more creative and professional layout, helping to effectively communicate ideas and milestones in a visually engaging way. However, for the Final Report, I switched to Microsoft Word, as it involved compiling all project components into a single, extensive document where structured formatting and easier content management were more suitable.

- **Microsoft Word**

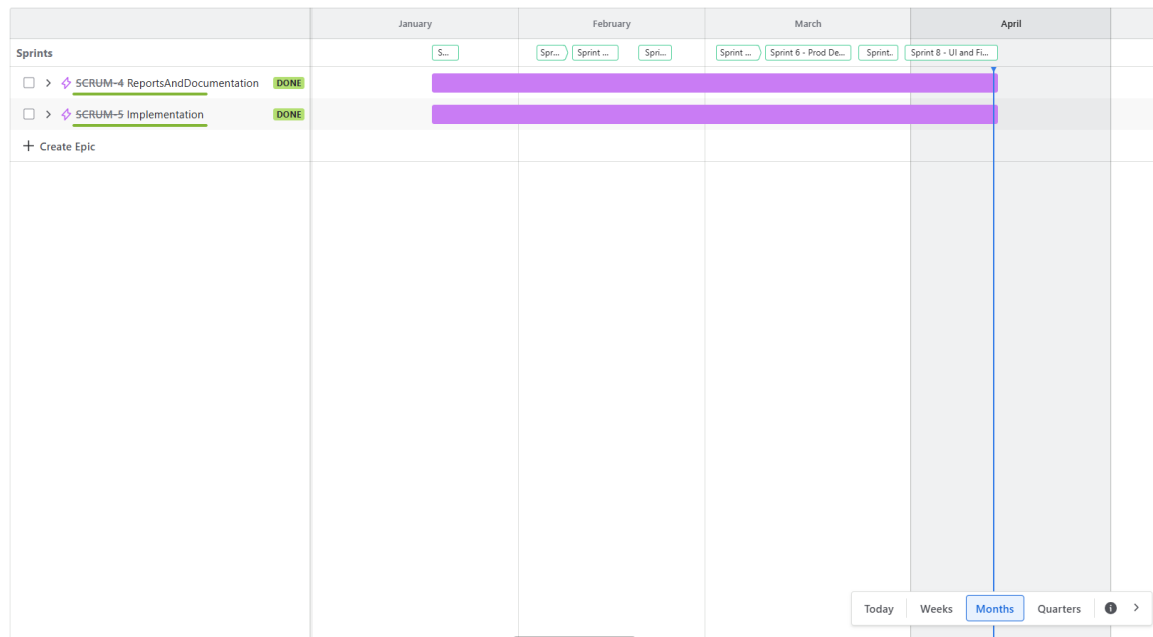
Used for compiling and formatting the Final Report.

Project Completion Timeline

System Architecture (MVC Pattern)



Gantt Chart



Sprint 1 (January 18 – 25, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
25-01-2025	SCRUM-1	Draft Project Proposal	6	Sprint 1 - FurBot Setup
19-01-2025	SCRUM-6	Set Up Project Repository	1	Sprint 1 - FurBot Setup
19-01-2025	SCRUM-2	Set Up JIRA for Project Management	2	Sprint 1 - FurBot Setup
18-01-2025	SCRUM-13	Research a topic for the project proposal	2	Sprint 1 - FurBot Setup
18-01-2025	SCRUM-7	Research and Identify Technologies for FurBot	2	Sprint 1 - FurBot Setup

Summary of Work Done

- **Drafted the Project Proposal (SCRUM-1)**
Researched and documented the scope, objectives, and deliverables for FurBot. Incorporated feedback from the professor to refine the direction of the project.
- **Set Up GitHub Repository (SCRUM-6)**
Initialized the GitHub repo to manage version control throughout the development cycle.
- **Created JIRA Board (SCRUM-2)**
Configured a JIRA board to manage tasks, track progress, assign story points (Work log), and maintain an agile workflow.
- **Conducted Initial Topic Research (SCRUM-13)**

Explored various approaches to pet-related chatbot solutions and framed the problem area to address a clear user need based on my own experience.

➤ **Identified and Finalized Technology Stack (SCRUM-7)**

Selected appropriate technologies including Flask, Vue.js, MongoDB Atlas.

Sprint 2 (February 3 – 8, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
06-02-2025	SCRUM-14	Test API	1	Sprint 2 - Set Up the Backend
06-02-2025	SCRUM-12	Build User Authentication API	3	Sprint 2 - Set Up the Backend
05-02-2025	SCRUM-11	Set Up MongoDB	1	Sprint 2 - Set Up the Backend
05-02-2025	SCRUM-10	Install Dependencies	1	Sprint 2 - Set Up the Backend
05-02-2025	SCRUM-9	Set Up Python and Virtual Environment	1	Sprint 2 - Set Up the Backend
05-02-2025	SCRUM-8	Create the Folder Structure in the Implementation folder	1	Sprint 2 - Set Up the Backend

Summary of Work Done

➤ **Created Backend Folder Structure (SCRUM-8)**

Organized the implementation folder with subdirectories for routes, services, and configurations, following a scalable Flask project structure.

➤ **Set Up Python Environment (SCRUM-9)**

Installed Python and configured a virtual environment to manage project dependencies in isolation.

➤ **Installed Required Dependencies (SCRUM-10)**

Installed essential libraries including Flask, Flask-JWT, Flask-Mail, pymongo, and FuzzyWuzzy.

➤ **Set Up MongoDB Integration (SCRUM-11)**

Connected the backend to MongoDB Atlas and verified connectivity for storing user and pet profile data.

➤ **Built the Authentication API (SCRUM-12)**

Developed the user authentication system using JWT, allowing secure login and registration functionality.

➤ **Tested API Endpoints (SCRUM-14)**

Verified core endpoints for authentication and data handling using tools like Postman to ensure stability and correct responses.

Sprint 3 (February 9 – 13, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
11-02-2025	SCRUM-20	Set up Job Scheduler to automate the email reminders	3	Sprint 3 - Backend Setup
09-02-2025	SCRUM-17	Set up Reminders	3	Sprint 3 - Backend Setup
09-02-2025	SCRUM-16	Create end points for pet profiles	2	Sprint 3 - Backend Setup
09-02-2025	SCRUM-15	Set up MongoDB for Pet Profiles	1	Sprint 3 - Backend Setup

Summary of Work Done

- **Set Up MongoDB for Pet Profiles (SCRUM-15)**
Created MongoDB collections and schemas to store pet-related data.
- **Create Endpoints for Pet Profiles (SCRUM-16)**
Developed API endpoints to support CRUD operations for pet profiles. Integrated routes with MongoDB to allow pet data creation, retrieval, updating, and deletion.
- **Set Up Reminders (SCRUM-17)**
Implemented the logic to schedule reminders for pet-related tasks (e.g., vaccinations, grooming). Designed backend logic to store and manage reminder data.
- **Set Up Job Scheduler to Automate Email Reminders (SCRUM-20)**
Integrated a job scheduler using APScheduler to send automated email notifications based on user-defined reminder dates. Emails are delivered to the user's registered email address using Flask-Mail.

Sprint 4 (February 14 – 18, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
18-02-2025	SCRUM-25	UI testing & debugging	6	Sprint 4 - Building Initial UI
18-02-2025	SCRUM-23	API integration for authentication & pet management	3	Sprint 4 - Building Initial UI
17-02-2025	SCRUM-24	User session handling (store token & authentication state)	4	Sprint 4 - Building Initial UI
17-02-2025	SCRUM-22	Basic UI for Adding & Viewing Pets	2	Sprint 4 - Building Initial UI
17-02-2025	SCRUM-21	Functional UI for Login & Sign-Up	2	Sprint 4 - Building Initial UI
06-02-2025	SCRUM-18	Progress Report #1	2	Sprint 4 - Building Initial UI

Summary of Work Done

- **Progress Report #1 (SCRUM-18)**
Compiled and submitted the first formal progress report covering Sprint 3 deliverables and initial plans for Sprint 4.
- **Initial Functional UI for Login & Sign-Up (SCRUM-21)**
Built the front-end components for user login and sign-up using Vue.js. Integrated form validation and navigation for smooth user flow.
- **Basic UI for Adding & Viewing Pets (SCRUM-22)**

Created the UI elements for users to input pet profiles and view existing entries for pet profile management.

- **User Session Handling (SCRUM-24)**
Implemented logic to store JWT tokens and maintain authentication state across pages using local storage.
- **API Integration for Authentication & Pet Management (SCRUM-23)**
Connected the frontend with backend API endpoints for user login, registration, and CRUD operations for pet profiles (e.g., POST to add user to the database and GET to display the saved pets).
- **UI Testing & Debugging (SCRUM-25)**
Performed testing on all UI components to fix layout issues, logic bugs, and ensure a responsive and smooth user experience.

Sprint 5 (February 19 – March 10, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
10-03-2025	SCRUM-44	Creating Progress Report #2	2	Sprint 5 - Chatbot Integration
06-03-2025	SCRUM-40	Integrate Yelp with Backend API - March 6	1	Sprint 5 - Chatbot Integration
06-03-2025	SCRUM-39	Integrate Yelp with Chatbot UI - March 6	1	Sprint 5 - Chatbot Integration
05-03-2025	SCRUM-38	Integrate Yelp with Backend API - March 5	1	Sprint 5 - Chatbot Integration
05-03-2025	SCRUM-37	Integrate Yelp with Chatbot UI - March 5	2	Sprint 5 - Chatbot Integration
05-03-2025	SCRUM-32	Implement Basic Conversation Flow	1	Sprint 5 - Chatbot Integration
04-03-2025	SCRUM-36	Integrate Yelp with Chatbot UI - March 4	2	Sprint 5 - Chatbot Integration
04-03-2025	SCRUM-35	Integrate Yelp with Backend API - March 4	2	Sprint 5 - Chatbot Integration
04-03-2025	SCRUM-30	Create initial Chatbot UI	1	Sprint 5 - Chatbot Integration
04-03-2025	SCRUM-28	Yelp Registration and Initial Set up	1	Sprint 5 - Chatbot Integration
03-03-2025	SCRUM-34	Integrate Yelp with Chatbot UI - March 3	1	Sprint 5 - Chatbot Integration
03-03-2025	SCRUM-31	Integrate Yelp with Backend API - March 3	1	Sprint 5 - Chatbot Integration
26-02-2025	SCRUM-27	Implement Basic Chatbot Logic	5	Sprint 5 - Chatbot Integration
20-02-2025	SCRUM-33	Midterm Report and Video	5	Sprint 5 - Chatbot Integration
19-02-2025	SCRUM-26	Create API Endpoint for Chatbot (/chatbot) and Simple UI Integration	5	Sprint 5 - Chatbot Integration

Summary of Work Done

- **Create API Endpoint for Chatbot (/chatbot) and Simple UI Integration (SCRUM-26)**
Built the initial Flask API endpoint for chatbot interaction. Set up a simple UI to test chatbot responses and ensure request-response flow using static intents.
- **Midterm Report and Video (SCRUM-33)**
Prepared and submitted a detailed midterm project report along with a walkthrough demo video highlighting progress and key features implemented.
- **Implement Basic Chatbot Logic (SCRUM-27)**

Developed the foundational logic for parsing user input and generating appropriate responses using FuzzyWuzzy string matching for intent detection and similarity scoring.

- **Yelp Registration and Initial Setup (SCRUM-28)**
Registered the app with Yelp Fusion API and configured API keys and request handling logic.
- **Integrate Yelp with Backend API (SCRUM-31, 35, 38, 40)**
Connected Yelp API to the backend by creating functions that dynamically fetch data for pet-related services based on user queries and location.
- **Create Initial Chatbot UI (SCRUM-30)**
Designed and implemented a basic chatbot interface allowing users to type messages and view responses in real-time.
- **Integrate Yelp with Chatbot UI (SCRUM-34, 36, 37, 39)**
Enhanced the chatbot frontend to handle and display Yelp results using cards to display the business name, address and ratings.
- **Implement Basic Conversation Flow (SCRUM-32)**
Created basic interaction logic including greetings, help prompts (e.g., what are the different questions user can ask, and fallback responses to improve user experience.
- **User Session Handling (SCRUM-24)**
Handled token storage and user session persistence to maintain login state across since I've implemented the dashboard.
- **API Integration for Authentication & Pet Management (SCRUM-23)**
Integrated both the chatbot and pet profile management into a unified dashboard. This required reworking the layout and logic to ensure seamless interaction between authentication, pet data handling, and chatbot functionality within a single interface.
- **Creating Progress Report #2 (SCRUM-44)**
Prepared report #2 for project tracking and submission.

Sprint 6 (March 11 – 18, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
18-03-2025	SCRUM-49	Deploy FurBot to Railway 3	6	Sprint 6 - Prod Deployment
17-03-2025	SCRUM-48	Deploy FurBot to Railway 2	4	Sprint 6 - Prod Deployment
12-03-2025	SCRUM-47	Test and Debug - March 12	4	Sprint 6 - Prod Deployment
12-03-2025	SCRUM-46	Deploy FurBot to Railway 1	4	Sprint 6 - Prod Deployment
11-03-2025	SCRUM-45	Sign up with Railway	2	Sprint 6 - Prod Deployment

Summary of Work Done

- **Sign Up with Railway (SCRUM-45)**
Created a Railway account and set up the project environment to enable cloud-based deployment for FurBot.
- **Deploy FurBot to Railway 1 (SCRUM-46)**
Performed the initial deployment of the FurBot backend to Railway.
- **Test and Debug – March 12 (SCRUM-47)**
Tested the deployed application, fixed issues related to API prefixing, CORS policies, and ensured that frontend-backend communication worked as expected.
- **Deploy FurBot to Railway 2 (SCRUM-48)**
Attempted a second round of deployment but continued facing CORS configuration issues that prevented the frontend from accessing the backend API. Investigated headers and Flask-CORS settings but was unable to fully resolve the issue during this sprint. Environment variable setup was completed.
- **Deploy FurBot to Railway 3 (SCRUM-49)**
Proceeded with the deployment despite ongoing CORS challenges. Updated wsgi.py and configured the production start command. Frontend assets loaded successfully, but API interaction remains limited due to unresolved CORS restrictions.

Sprint 7 (March 24 – 30, 2025)

Date Worked On	Issue key	Summary	Hours Logged	Sprint
30-03-2025	SCRUM-54	Progress Report #4	1	Sprint 7 - Auto Email & UI
28-03-2025	SCRUM-53	Retry Production Environment Deployment	6	Sprint 7 - Auto Email & UI
26-03-2025	SCRUM-58	UI Enhancement 2	1.5	Sprint 7 - Auto Email & UI
26-03-2025	SCRUM-29	Store Frequently Asked Questions in MongoDB	3	Sprint 7 - Auto Email & UI
25-03-2025	SCRUM-55	UI Enhancement 1	4	Sprint 7 - Auto Email & UI
24-03-2025	SCRUM-52	Automate Email Reminders	2	Sprint 7 - Auto Email & UI

Summary of Work Done

- **Automate Email Reminders (SCRUM-52)**

Refined the job scheduler to ensure reminder emails are sent using PST (Vancouver Time). Verified backend logic for triggering emails and improved reliability of the APScheduler task.

- **UI Enhancement 1 (SCRUM-55)**
Changes the theme into green as light and family and pet-oriented app.
- **UI Enhancement 2 (SCRUM-58)**
Continuation of UI enhancement.
- **Store Frequently Asked Questions in MongoDB (SCRUM-29)**
Added a new MongoDB collection to store FAQs for chatbot use. This enables static responses to common user queries and lays the groundwork for richer conversational flows e.g., adding spaCy.
- **Retry Production Environment Deployment (SCRUM-53)**
Made another attempt to deploy FurBot to Railway. Continued efforts to resolve CORS issues and verify backend availability in production, with partial success.
- **Progress Report #4 (SCRUM-54)**
Documented the sprint's progress, challenges (particularly deployment-related), and updates to features like email reminders and UI enhancements.

Sprint 8 (March 29, - April 13, 2025)

Date Worked On	Summary	Hours Logged	Sprint
13-04-2025 0:00	Presentation Preparation		Sprint 8 - UI and Final Report
12-04-2025 0:00	Final Report Documentation 5	8	Sprint 8 - UI and Final Report
11-04-2025 0:00	Final Report Documentation 4	5	Sprint 8 - UI and Final Report
10-04-2025 0:00	Final Report Documentation 3	3	Sprint 8 - UI and Final Report
09-04-2025 0:00	Final Report Documentation 2	2	Sprint 8 - UI and Final Report
08-04-2025 0:00	spaCy Implementation for location extraction	5	Sprint 8 - UI and Final Report
07-04-2025 0:00	User Evaluation	2	Sprint 8 - UI and Final Report
07-04-2025 0:00	Personalize Feature for additional novelty Day 4	2	Sprint 8 - UI and Final Report
06-04-2025 0:00	Progress Report #5	1.5	Sprint 8 - UI and Final Report
04-04-2025 0:00	Personalize Feature for additional novelty Day 3	2.5	Sprint 8 - UI and Final Report
03-04-2025 0:00	Personalize Feature for additional novelty Day 2	2.5	Sprint 8 - UI and Final Report
02-04-2025 0:00	Final Report Documentation 1	1	Sprint 8 - UI and Final Report
01-04-2025 0:00	Personalize Feature for additional novelty Day 1	3	Sprint 8 - UI and Final Report

Summary of Work Done:

- **Final Report Documentation 1–5 (SCRUM-62 to SCRUM-66)**
Prepared the comprehensive final report in five stages, covering all technical components of FurBot. Sections included: system overview, implemented features, backend logic, API integrations (spaCy, Yelp), changes to the proposal and project completion timeline.

- **spaCy Implementation for Location Extraction (SCRUM-60)**
Integrated spaCy to extract city/location names from chatbot queries. This enhanced the chatbot's ability to deliver accurate, location-specific responses for pet services (e.g., "dog groomers in Burnaby").
- **User Evaluation (SCRUM-61)**
Conducted user testing to assess ease of use, chatbot interaction quality, and overall satisfaction. Collected feedback on reminder creation, pet profile setup, and chatbot suggestions for future improvements.
- **Personalize Feature for Additional Novelty – Day 1 to 4 (SCRUM-56 to SCRUM-59)**
Implemented FurBot's intelligent reminder suggestion system.
 - Tracked last sent reminder date per pet
 - Suggested reminders like grooming or vaccination based on time elapsed
 - Integrated logic into chatbot to offer actionable promptsThis feature adds novelty and proactive care suggestions for users.
- **Progress Report #5 (SCRUM-67)**
Compiled and submitted the fifth progress report. Summarized feature implementations, ongoing challenges (e.g., spaCy accuracy, UI polish), and deployment readiness.
- **Presentation Preparation (SCRUM-68)**
Developed slides and demo plan for the final presentation. Ensured feature walkthroughs (chatbot, reminders, favorites) were well-organized and aligned with project goals. Practiced delivery to meet time constraints.

Implemented Features

Implemented Feature 1: Login

Description:

This feature allows registered users to securely log in to the FurBot platform. The frontend login form captures the user's email and password, then sends a request to the backend API. Upon successful authentication, a JWT token and user ID are returned and stored in localStorage, granting access to the dashboard and other authenticated features.

Frontend (Vue.js): LoginPage.vue

Handle user login through a form that collects email and password. On submission.

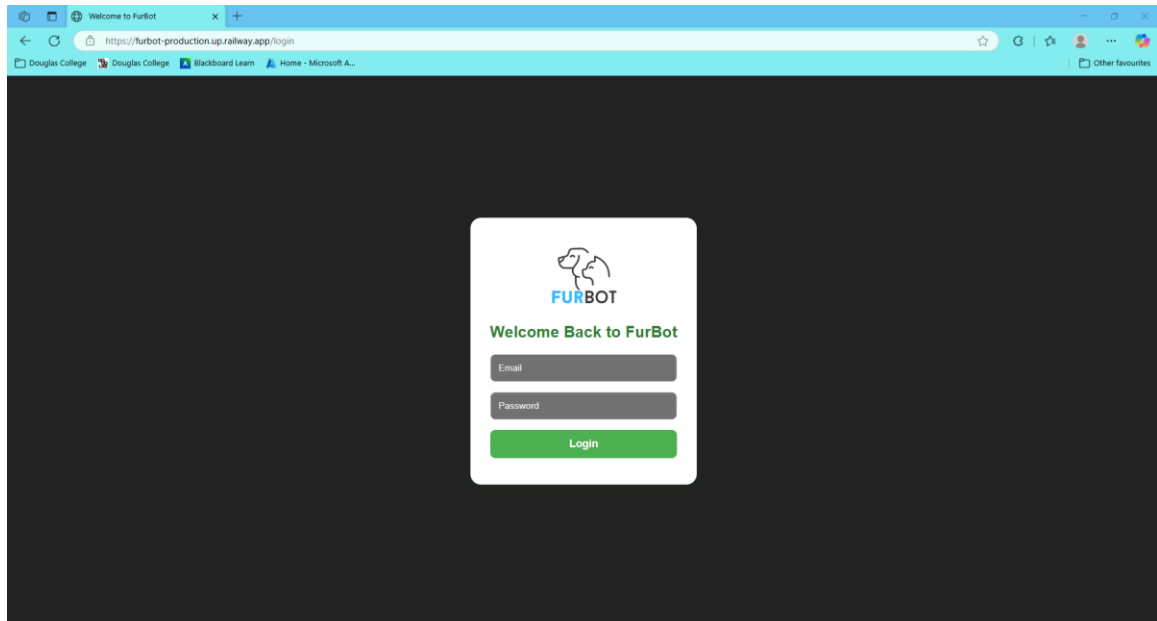


Figure 0: Log in UI with form.

Code Highlight: LoginPage.vue - **handleLogin()** method calls the **authService.js** to validate the credentials and store the token

```
async handleLogin() {
  try {
    const response = await login(this.email, this.password); // Call the Login function from authService
    localStorage.setItem("token", response.token); // Store the token in localStorage
    localStorage.setItem("user_id", response.user_id); // Store the user ID in localStorage
    this.$router.push("/dashboard"); // Redirect to the dashboard after successful login
  } catch (error) {
    this.errorMessage = "Login failed. Please check your credentials."; // Set the error message if login fails
  }
},
```

Figure 1: Method from LoginPage.vue triggering API Call and redirecting on success

Service Layer: Contains the login() function, which sends a POST request to the Flask API with the user's credentials. It expects a response containing a token and user ID.

```
// Login function for POST request.
export const login = async (email, password) => {
  try {
    const response = await axios.post(`${API_URL}/login`, { email, password }, { // Send Login data in the request body
      headers: { "Content-Type": "application/json" }, // Proper header for JSON content
      withCredentials: true,
    });

    if (response.data.token && response.data.user_id) { // Check if token and user_id are present in the response
      return response.data; // Return the response data containing token and user_id
    } else {
      throw "Invalid Login Response"; // If the response does not contain token and user_id, throw an error
    }
  } catch (error) {
    console.error("Login Error:", error);
    throw error.response?.data?.error || "Login failed";
  }
};
```

Figure 2: Login function in authService.js that handles API call and returns token upon success

Backend: (/api/login) This Flask route handles authentication. It validates the email and password against the MongoDB user's collection. If valid, it generates a signed JWT token valid for one hour and returns it with the user ID.

```
# -----
# API: User Login
# -----
@app.route(f"{API_PREFIX}/login", methods=["POST", "OPTIONS"])
@cross_origin(origins=allowed_origins, supports_credentials=True, methods=["POST", "OPTIONS"])
def login():
    # Handle preflight request
    if request.method == "OPTIONS":
        return '', 200 # Respond with 200 OK for preflight request

    try:
        # Parse the incoming JSON data from the request body
        data = request.get_json()

        # Validate that both 'email' and 'password' are provided in the request
        if not data or "email" not in data or "password" not in data:
            return jsonify({"error": "Invalid request, missing email or password"}), 400

        # Look up the user in the MongoDB 'users' collection by email
        user = users.find_one({"email": data["email"]})

        # If no user found or password doesn't match, return an error
        if not user or not check_password_hash(user["password"], data["password"]):
            return jsonify({"error": "Invalid credentials"}), 401

        # Generate a JWT token (valid for 1 hour)
        token = jwt.encode(
            {"email": data["email"], "exp": datetime.datetime.now(datetime.timezone.utc) + datetime.timedelta(hours=1)},
            app.config["SECRET_KEY"], # Use Flask App secret key for signing the token
            algorithm="HS256", # Use HMAC SHA-256 algorithm for signing
        )

        # Return the token and user ID in the response
        return jsonify({"token": token, "user_id": str(user["_id"])}), 200

    except Exception as e:
        # If any error occurs during the process, log it and return a server error response
        print("Login error:", str(e))
```

Figure 3: Flask API endpoint for login, handling credential validation and token generation

Summary of Logic Flow:

1. User enters email and password in the login form
2. LoginPage.vue triggers handleLogin() calls authService.js - login().
3. authService.js sends POST to /api/login.

4. Flask verifies credentials and return JWT + user ID.
5. Token and user ID are stored in localStorage (e.g., cookies).
6. Users redirected to the dashboard.

Implemented Feature 2: Registration

Description:

This feature enables new users to create an account in FurBot by entering their email, and password. Once successfully registered, the user is redirected to the login page to access personalized features.

Frontend (Vue.js): SignupPage.vue

Displays a registration form that collects the user's email, and password. On submission, the `handleRegister()` method calls the `register()` function in `authService.js`, handles validation, and redirects the user upon success.

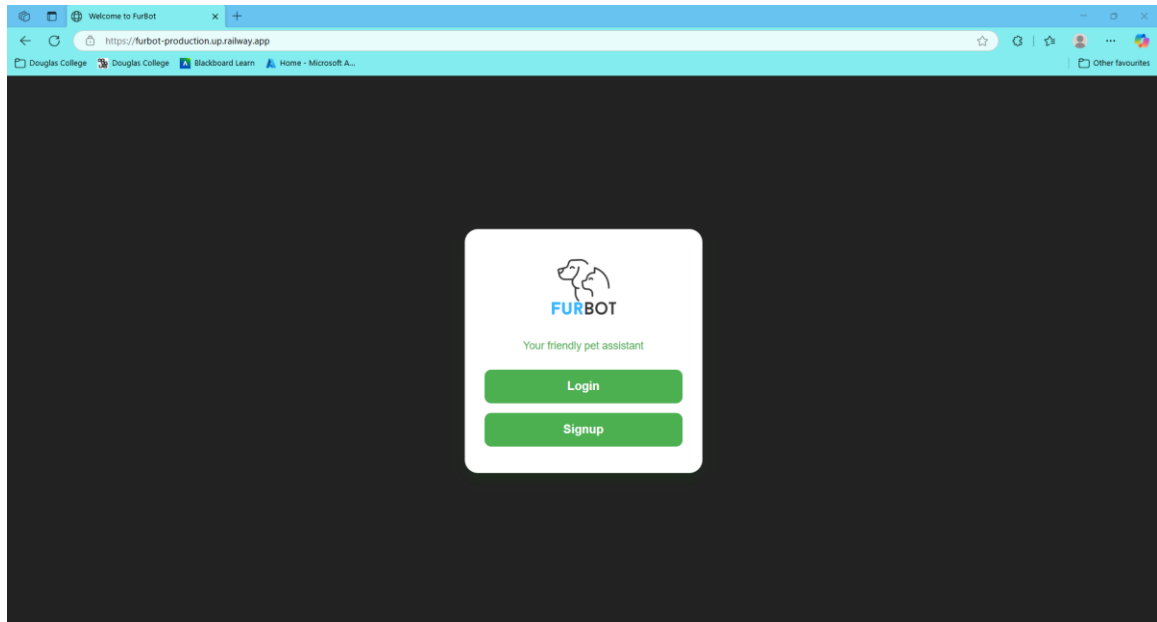


Figure 4: Sign up UI with form

Code Highlight: SignupPage.vue - **handleSignup()** method calls the register() function in **authService.js**, handles validation and redirects upon successful.

```
async handleSignup() {
  try {
    const response = await signup(this.email, this.password); // Call the signup function from authService
    alert(response.message || "Signup successful!"); // Show success message

    // Redirect to login page after successful signup
    this.$router.push("/login");
  } catch (error) {
    this.errorMessage = error.message || "An error occurred during signup.";
    alert(this.errorMessage);
  }
},
```

Figure 5: handleSignup() method from SignupPage.vue handling user registration and form submission

Service Layer: Defines the signup() method which sends a POST request to the /api/register endpoint. It returns the response message or throws an error if validation fails.

```
// Signup function for POST request.
export const signup = async (email, password) => {
  try {
    const response = await axios.post(`${API_URL}/register`, { email, password }, { // Send signup data in the request body
      headers: {
        "Content-Type": "application/json", // Proper header for JSON content
      },
    });
    return response.data;
  } catch (error) {
    console.error("Signup Error:", error);
    throw error.response?.data?.error || "Signup failed";
  }
};
```

Figure 6: Registration method in authService.js that sends POST request to app.py (backend)

Backend: (/api/register) This API endpoint validates input data, hashes the password securely using werkzeug.security, and inserts the new user into the MongoDB users collection. If the user already exists, it returns an error.

```
# -----
# API: User Registration
# -----

@app.route(f"{API_PREFIX}/register", methods=["POST"])
def register():
    data = request.json

    # Check if email already exists
    if users.find_one({"email": data["email"]}):
        return jsonify({"error": "Email already exists"}), 400

    # Hash the password using werkzeug.security before storing to mongoDB
    hashed_password = generate_password_hash(data["password"])
    users.insert_one({"email": data["email"], "password": hashed_password})
    return jsonify({"message": "User registered successfully!"}), 201
```

Figure 7: Flask API route for /api/register handling input validation and user creation

Database: users collection

furbot.users

STORAGE SIZE: 36KB LOGICAL DATA SIZE: 2.53KB TOTAL DOCUMENTS: 11 INDEXES TOTAL SIZE: 36KB

[Find](#) [Indexes](#) [Schema Anti-Patterns](#) [Aggregation](#) [Search Indexes](#)

[Generate queries from natural language in Compass](#)

[Filter](#) Type a query: { field: 'value' }

QUERY RESULTS: 1-11 OF 11

```

_id: ObjectId('67b03b5c48eb886e4bb1b889')
email: "cm44228@gmail.com"
password: "scrypt:32768:8:1$eobswDxGzyBmgAOZ$caad0d7115270b19c89315ab01ecb6b6fb0b..."

_id: ObjectId('67e7a1f6943842ab099c800a')
email: "charlz1202@yahoo.com"
password: "scrypt:32768:8:1$PtDUvG06pNqYcKLu$942cc7b8866f192c74cd60015478d30ea369..."

```

Summary of Logic Flow:

1. User enters name, email, and password in the registration form.
2. SignupPage.vue triggers handleRegister() calls authService.js - register().
3. authService.js sends POST to /api/register.
4. Flask validates input, hashes the password, saves to MongoDB.
5. Success response is returned and user is redirected to /login.

Implemented Feature 3: Dashboard

Description:

After successful login, users are redirected to the Dashboard, where they can manage their pet-related information. This includes viewing and managing favorites, adding or deleting pets, checking upcoming reminders, and interacting with the FurBot Chatbot for assistance. The interface is personalized for the logged-in user using their JWT token and user ID stored in localStorage.

Frontend (Vue.js): Dashboard.vue

- <FavoriteList /> – Displays saved result from Yelp.
- <PetList /> – Shows all pets registered by the user
- <ReminderList /> – Lists upcoming pet care reminders
- <Chatbot/> – AI chatbot that supports pet-related queries and integrates with Yelp.
- Logout button (calls handleLogout())

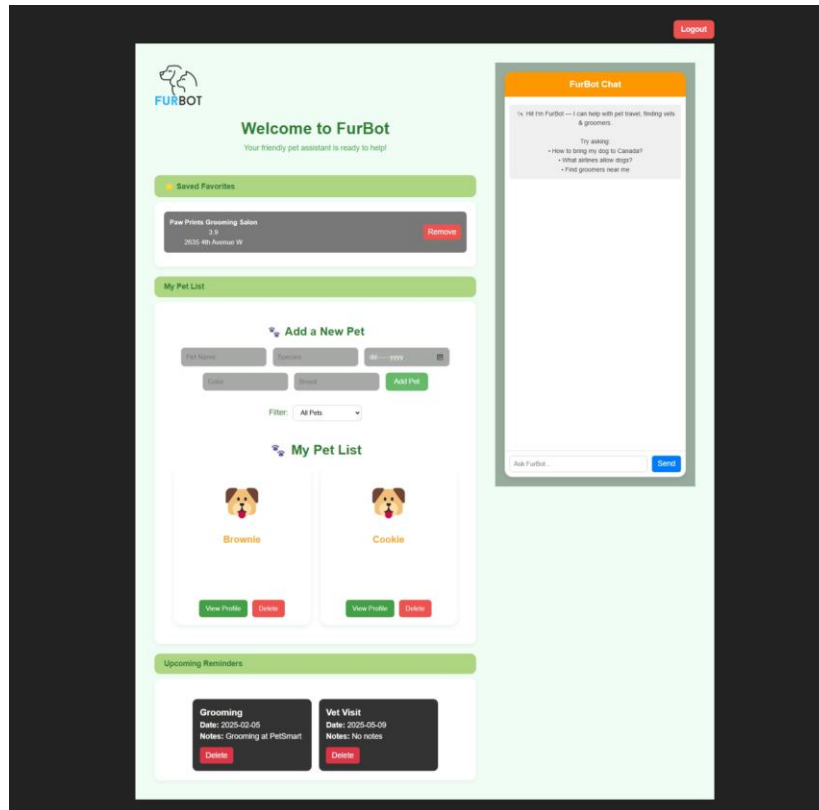


Figure 8: Dashboard UI and chatbot where user can input a question.

Code Highlight: Dashboard.vue - `handleLogout()` Logs the user out and redirects to the login page.

```
async handleLogout() {
  try {
    await logout();
    this.$router.push("/login");
  } catch (error) {
    console.error("Logout failed:", error);
  }
},
```

Figure 9: `handleLogout()` method from Dashboard.vue logs the user out of FurBot and redirected to Login page.

JWT Token Check (on mount):

```
mounted() {
  // Check if user is logged in, if not redirect to login page
  const token = localStorage.getItem("token");
  if (!token) {
    this.$router.push("/login");
  }
},
```

Figure 10: Dashboard logic in Dashboard.vue checking for authentication

Summary of Logic Flow:

1. After login, user is redirected to /dashboard
2. Vue checks for JWT token in localStorage
3. Displays:
 - User's favorite businesses
 - Pet profiles from MongoDB (/api/pets)
 - Reminders fetched via /api/reminders
 - Chatbot available for pet related queries

Implemented Feature 4: Pet Profile Management

Description:

This feature allows users to manage their pets from the Dashboard after login. Users can add a new pet, view all registered pets, and remove any pet from the list. Each pet includes attributes like name, species, birthdate, breed, and color. The data is saved in the MongoDB collection and retrieved using the logged-in user's ID (via JWT token).

Frontend (Vue.js): PetList.vue

Add a Pet Form

User can input a new pet's details and submit the form. The data is sent to the backend using Axios and displayed immediately in the pet list.

Pet List Display

Displays all pets saved by the current user in card format. Each card includes the pet's name, species, and action buttons (View Profile, Delete).

The screenshot displays a web interface titled "My Pet List" with a light green header. Below the header, there is a section for "Add a New Pet" featuring a paw print icon and a form with five input fields: "Pet Name", "Species", a date field labeled "dd-----yyyy" with a calendar icon, "Color", and "Breed". A green "Add Pet" button is positioned to the right of the "Breed" field. Below the form is a "Filter:" dropdown menu currently set to "All Pets". Underneath the filter, the section "My Pet List" is shown with a paw print icon. It contains two pet cards. Each card features a dog icon, the pet's name ("Brownie" and "Cookie" respectively), and two buttons at the bottom: "View Profile" (green) and "Delete" (red).

Figure 11: Add a Pet Form UI from PetList.vue

Code Highlight: Displays a form for adding a new pet and a list of pets saved by the logged-in user. The addNewPet() method is triggered when the form is submitted. It calls the addPet() method in the service layer, cleans the form, and refreshes the pet list display fetchPets() method and it triggers getPets() from service layer. Also, if user delete a pet, it calls the deletePet() method which then calls deletePet() from service layer.

```

async fetchPets() {
    const ownerId = localStorage.getItem("user_id");
    if (!ownerId) throw new Error("User not logged in");
    this.pets = await getPets(ownerId);
} catch (error) {
    console.error("Error fetching pets: ", error);
}

// addNewPet method is called when the form is submitted
async addNewPet() {
    try {
        if (!this.ownerId) throw new Error("User not logged in");

        // Trigger the addPet function from petService.js to add a new pet passing the required parameters
        await addPet(
            this.ownerId,
            this.name,
            this.species,
            this.dob,
            this.color,
            this.breed
        );

        // Reset the form fields after successful addition of pet
        this.name = "";
        this.species = "";
        this.dob = "";
        this.color = "";
        this.breed = "";

        alert("Pet added successfully"); // Show success message
        await this.fetchPets(); // Fetch the updated list of pets
    } catch (error) {
        console.error("Error adding pet: ", error);
        alert("Failed to add pet");
    }
}

async deletePet(petId) {
    try {
        await deletePet(petId);
        this.pets = this.pets.filter((pet) => pet._id !== petId);
    } catch (error) {
        console.error("Error deleting pet: ", error);
    }
}

```

Figure 12: Method in PetService.js handling form submission for adding a new pet and displaying pets and it also allow user to delete a pet.

Service Layer: addPet() in petServices.js – This function sends a POST request to the /api/pets endpoint with the pet data and the user's ID. It uses the authentication token stored in localStorage to associate the pet with the correct user.

```
export const getPets = async (ownerId) => {
  try {
    const token = localStorage.getItem("token");
    const response = await axios.get(`${API_URL}/pets/${ownerId}`, {
      headers: {Authorization: `Bearer ${token}`},
    });

    console.log("Pets Response from Flask:", response.data);

    return response.data;
  } catch (error) {
    console.error("Get Pets Error:", error);
    throw error.response?.data?.error || "Failed to fetch pets";
  }
};

// This function is used to add a pet to the database. It takes the owner's ID, pet's name, species, date of birth, color, and breed as parameters.
export const addPet = async (ownerId, petName, species, dob, color, breed) => {
  try {
    const token = localStorage.getItem("token");

    if(!ownerId) {
      throw new Error("Owner ID is required to add a pet");
    }

    const payload = {
      owner_id: String(ownerId),
      name: petName,
      species: species || "",
      dob: dob,
      color: color || "",
      breed: breed || "",
    };

    console.log("Sending payload to Flask:", JSON.stringify(payload, null, 2)); // Debugging step

    const response = await axios.post(`${API_URL}/pets`, payload, {
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
    });

    return response.data;
  } catch (error) {
    console.error("Add Pet Error:", error);
  }
};

export const deletePet = async (petId) => {
  try {
    const token = localStorage.getItem("token");
    const response = await axios.delete(`${API_URL}/pets/${petId}`, {
      headers: {
        Authorization: `Bearer ${token}`,
      },
    });

    return response.data;
  } catch (error) {
    console.error("Delete Pet Error:", error);
    throw error.response?.data?.error || "Failed to delete pet";
  }
};
```

Figure 13: Method in petService.js that sends a POST | GET | DELETE request with pet data to the backend

Backend: (/api/pets) This endpoint receives the POST request to create a pet. It verifies the user's token, saves the pet to the pets collection in MongoDB, and associates it with the provided owner_id.

```
# -----
# API: Add a new pet
# -----
@app.route(f"{API_PREFIX}/pets", methods=["POST"])
def add_pet():
    data = request.json

    owner_id = str(data.get("owner_id", "")) # Get the owner_id from the request data
    # Check if the owner exists

    # Check if required fields are present
    if not owner_id or "name" not in data or "dob" not in data:
        return jsonify({"error": "Missing required fields (owner_id, name, dob)"}), 400

    # Validate the owner_id format
    try:
        owner_obj_id = ObjectId(owner_id)
    except Exception:
        return jsonify({"error": "Invalid owner_id format"}), 400

    # Check if pet already exists for the owner
    existing_pet = pets.find_one({"owner_id": owner_obj_id, "name": data["name"]})
    if existing_pet:
        return jsonify({"error": "A Pet with this name already exists for the owner"}), 400

    # Insert pet into the database
    pet_id = pets.insert_one({
        "owner_id": owner_obj_id,
        "name": data["name"],
        "species": data.get("species", ""),
        "dob": data["dob"], # (format: YYYY-MM-DD)
        "color": data.get("color", ""),
        "breed": data.get("breed", ""),
        "medical_history": data.get("medical_history", []),
        "created_at": datetime.datetime.now(datetime.timezone.utc)
    }).inserted_id

    return jsonify({"message": "Pet added successfully", "pet_id": str(pet_id)}), 201

# -----
# API: Get all pets of an owner
# -----
@app.route(f"{API_PREFIX}/pets/<owner_id>", methods=["GET"])
def get_pets(owner_id):
    try:
        owner_obj_id = ObjectId(owner_id) # Validate owner_id format
    except Exception:
        return jsonify({"error": "Invalid owner_id format"}), 400

    pets_list = list(pets.find({"owner_id": owner_obj_id}, {"_id": 1, "name": 1, "species": 1, "breed": 1, "dob": 1, "color": 1, "medical_history": 1}))

    if not pets_list:
        return jsonify({"message": "No pets found for the owner"}), 404

    # Convert ObjectId to string before returning the response
    today = datetime.date.today()
    for pet in pets_list:
        pet["_id"] = str(pet["_id"])
        # Calculate the age of the pet
        if "dob" in pet:
            dob = datetime.datetime.strptime(pet["dob"], "%Y-%m-%d").date()
            age = today.year - dob.year
            if (today.month, today.day) < (dob.month, dob.day):
                age -= 1 # Subtract if the birthday hasn't occurred yet
            pet["age"] = age
        else:
            pet["age"] = "Unknown"
    return jsonify(pets_list), 200

# -----
# API: Delete a pet
# -----
@app.route(f"{API_PREFIX}/pets/<pet_id>", methods=["DELETE"])
def delete_pet(pet_id):
    try:
        ObjectId(pet_id) # Validate pet_id format
    except Exception:
        return jsonify({"error": "Invalid pet_id format"}), 400

    # Check if pet exists
    pet = pets.find_one({"_id": ObjectId(pet_id)})
    if not pet:
        return jsonify({"error": "Pet not found"}), 404

    pets.delete_one({"_id": ObjectId(pet_id)})
    return jsonify({"message": "Pet deleted successfully"}), 200
```

Figure 14: Flask API route for /api/pets (POST) , /api/pets/<owner_id> , /api/pets/<pet_id> handling input validation and user creation

Database: Pets collection from MongoDB

furbot.pets
 STORAGE SIZE: 36KB LOGICAL DATA SIZE: 5.32KB TOTAL DOCUMENTS: 22 INDEXES TOTAL SIZE: 36KB

Find Indexes Schema Anti-Patterns Aggregation Search Indexes

Generate queries from natural language in Compass

Filter Type a query: { field: 'value' }

QUERY RESULTS: 1-20 OF MANY

```

_id: ObjectId('67a950ed4b85a41a5e40b9cf')
owner_id: ObjectId('67a93766550cfe442245420b')
name: "Churros"
species: "Dog"
dob: "2019-02-5"
color: ""
breed: "Poodle"
medical_history: Array (1)
created_at: 2025-02-10T01:05:49.505+00:00
lastReminders: Object

_id: ObjectId('67b0569502d39aa21516807b')
owner_id: ObjectId('67b03b5c48eb886e4bb1b889')
name: "Buddy"
species: "Dog"
dob: "2021-02-15"
```

Figure 15: MongoDB Pets collection

Summary of Logic Flow:

Add new pet:

1. User enters pet details into the form on PetList.vue
2. addNewPet() triggers → calls addPet() from the service layer
3. addPet() sends a POST request to /api/pets with JWT token and form data
4. Flask validates the token and saves the pet in MongoDB under the user's owner_id
5. The updated pet list is fetched using getPets() and reloaded in the dashboard

To retrieve pets:

6. In PetList.vue, fetchPets() is triggered calls getPets() using the logged-in user's ID
7. getPets() sends a GET request to /api/pets, including the token
8. Flask decodes the token and returns only the pets associated with the authenticated owner_id
9. The frontend displays the fetched pet list in real-time

To delete a pet:

10. User clicks the “Delete” button on a pet card
11. deletePet() is called → sends a DELETE request to /api/pets/<pet_id>
12. Flask verifies the token and removes the pet from MongoDB
13. The pet list is refreshed by calling getPets() again and updating the view

Implemented Feature 5: Pet Care Reminder and Scheduling

Description:

This feature allows users to manage care reminders for their pets, such as grooming, vaccinations, and vet visits. From the pet profile screen, users can view, add, and delete reminders associated with a specific pet. All reminder data is stored in the MongoDB reminders collection and is only accessible to authenticated users.

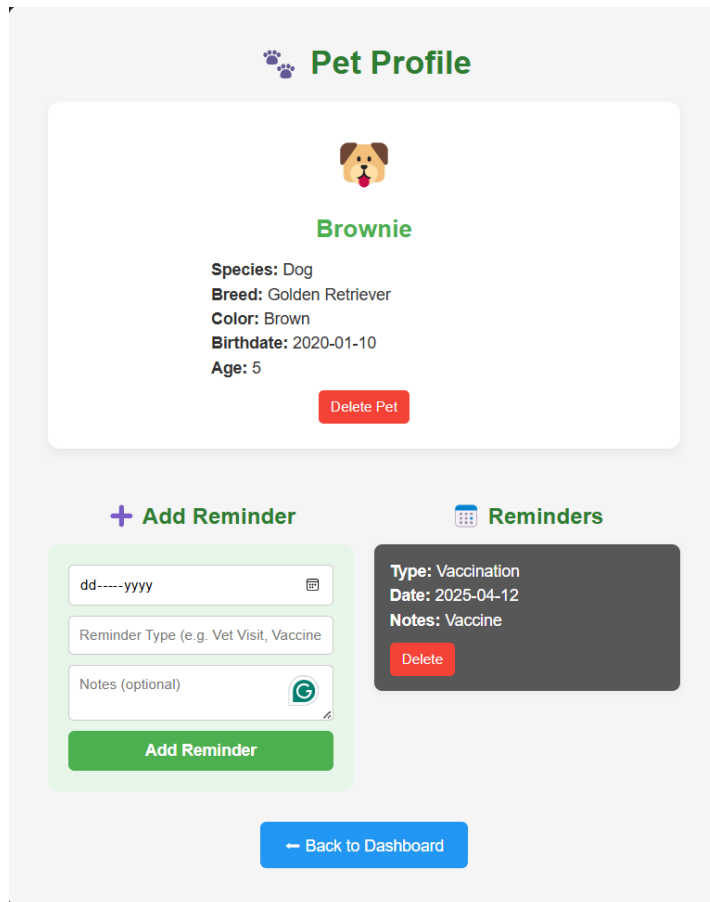
Frontend (Vue.js): PetProfile.vue

Add a Reminder Form

User can input a new pet's reminder (e.g., Vaccine, Grooming or Vet Visit) and submit the form. The data is sent to the backend using Axios and displayed immediately in the PetProfile.vue.

Reminder Display

Displays all reminders saved by the current pet in card format. Each card includes the specific pet's reminder.



The screenshot shows a web application interface for a pet profile. At the top, there's a header with a paw print icon and the text "Pet Profile". Below this, a white card displays a dog emoji, the name "Brownie" in green, and pet details: "Species: Dog", "Breed: Golden Retriever", "Color: Brown", "Birthdate: 2020-01-10", and "Age: 5". A red "Delete Pet" button is at the bottom right of the card. Below the card, there are two main sections. On the left, a green-bordered box titled "+ Add Reminder" contains a date input field (dd-....-yyyy), a text input for "Reminder Type (e.g. Vet Visit, Vaccine)", a text input for "Notes (optional)" with a green circular icon, and a green "Add Reminder" button. On the right, a dark gray box titled "Reminders" with a calendar icon shows a single reminder: "Type: Vaccination", "Date: 2025-04-12", "Notes: Vaccine", and a red "Delete" button. At the bottom center, there is a blue button with a left arrow and the text "Back to Dashboard".

Figure 16: Add reminder and fetchReminders() to get the list of reminders for a pet from PetProfile.vue

Reminder Display

The mounted() lifecycle hook in ReminderList.vue is triggered when Dashboard.vue is loaded, and it calls getRemindersForOwner() to fetch reminders

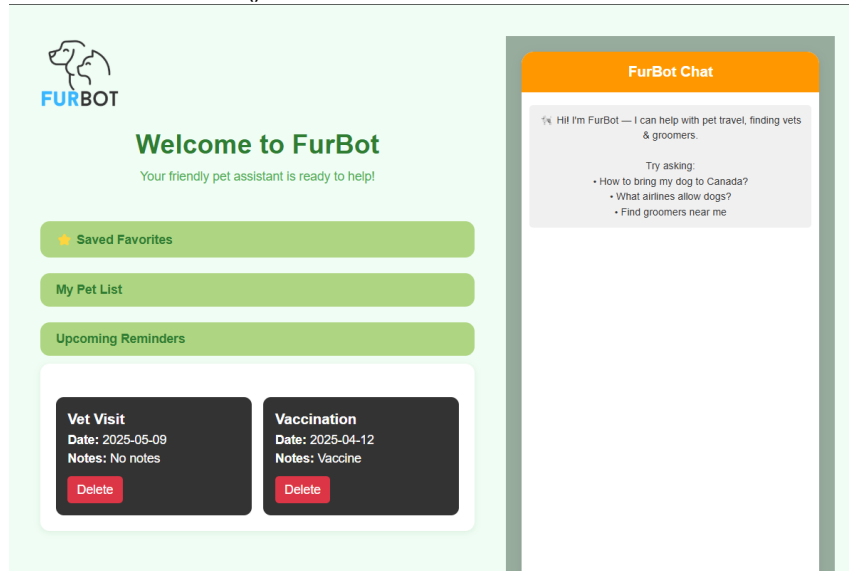


Figure 17: Reminder list from Dashboard which gets the reminder for an owner.

Code Highlight: The component fetches reminders for a specific pet when the page loads. It also includes a form where users can input a reminder type (e.g., grooming), choose a date, and optionally add a note.

- When the form is submitted, the addNewReminder() method calls the addReminder() function from the service layer, then refreshes the reminder list.

```
async addNewReminder() {
  const petId = this.$route.params.id;
  const ownerId = localStorage.getItem('user_id');
  await addReminder(ownerId, petId, this.newReminder.date, this.newReminder.type, this.newReminder.notes);
  this.newReminder = { date: '', type: '', notes: '' };
  await this.fetchReminders();
},
```

Figure 18: addNewReminder() method from PetProfile.vue.

- In PetProfile.vue, the fetchReminders() method invokes getRemindersForPet() from the service layer, which sends a GET request to /api/reminders/<pet_id> to retrieve pet-specific reminders.

```
async fetchReminders() {
  const petId = this.$route.params.id;
  if (petId) {
    this.reminders = await getRemindersForPet(petId);
  }
},
```

Figure 19: fetchReminders() method from PetProfile.vue

- In Dashboard.vue, the mounted() lifecycle hook of the embedded ReminderList.vue component is triggered automatically. Inside ReminderList.vue, it calls getRemindersForOwner(), which sends a GET request to /api/reminders/owner/<owner_id> to retrieve owner-wide reminders.

```
async mounted() {
  try {
    const ownerId = localStorage.getItem('user_id');
    if(!ownerId) {
      throw new Error("User ID not found");
    }

    if(this.petID) {
      this.reminders = await getRemindersForPet(this.petID);
      console.log("Fetched Pet Reminders: ",this.reminders);
    } else {
      this.reminders = await getRemindersForOwner(ownerId);
      console.log("Fetched Owner Reminders: ",this.reminders);
    }
  } catch (error) {
    console.error("Error fetchin reminders: ",error);
  }
},
```

Figure 20: mounted() lifecycle embedded in ReminderList.vue.

- The deleteReminder() calls deleteReminder() from the service layer that sends a DELETE request from the backend /api/reminders/<reminderId> to remove the reminder.

```
async deleteReminder(reminderId) {
  await deleteReminder(reminderId);
  this.reminders = this.reminders.filter(r => r._id !== reminderId);
},
goToDashboard() {
  this.$router.push('/dashboard');
}
```

Figure 21: deleteReminder from PetProfile.vue.

Service Layer: addPet() in petServices.js – This function sends a POST request to the /api/pets endpoint with the pet data and the user's ID. It uses the authentication token stored in localStorage to associate the pet with the correct user.

```
export const addReminder = async (ownerId, petId, date, type, notes) => {
  try {
    const token = localStorage.getItem("token");
    if (!token) throw new Error("User not authenticated. Please log in.");
    if (!ownerId || !petId || !date) throw new Error("Missing required fields for reminder.");

    // Fix date YYYY-MM-DD format
    const datePattern = /^\\d{4}-\\d{2}-\\d{2}$/;
    if (!datePattern.test(date)) throw new Error("Invalid date format. Use YYYY-MM-DD.");

    const payload = { owner_id: ownerId, pet_id: petId, date, type, notes };
    console.log("Sending Reminder:", JSON.stringify(payload, null, 2));

    const response = await axios.post(`${API_URL}/reminders`, payload, {
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${token}`,
      },
    });

    console.log("Reminder Added:", response.data);
    return response.data;
  } catch (error) {
    console.error("Add Reminder Error:", error);
    throw error.response?.data?.error || "Failed to add reminder";
  }
};

export const getRemindersForPet = async (petId) => {
  try {
    const token = localStorage.getItem("token");
    const response = await axios.get(`${API_URL}/reminders/${petId}`, {
      headers: { Authorization: `Bearer ${token}` },
    });

    console.log("Reminders Response (Pet):", response.data);
    return response.data;
  } catch (error) {
    console.error("Get Reminders Error:", error.response?.data || "No Response");
    throw error.response?.data?.error || "Failed to fetch reminders for pet";
  }
};

export const getRemindersForOwner = async (ownerId) => {
  try {
    const token = localStorage.getItem("token");
    const response = await axios.get(`${API_URL}/reminders/owner/${ownerId}`, {
      headers: { Authorization: `Bearer ${token}` },
    });

    console.log("Reminders Response (Owner):", response.data);
    return response.data;
  } catch (error) {
    console.error("Get Reminders Error:", error.response?.data || "No Response");
    throw error.response?.data?.error || "Failed to fetch reminders for owner";
  }
};

export const deleteReminder = async (reminderId) => {
  try {
    const token = localStorage.getItem("token");
    console.log(`Deleting Reminder with ID: ${reminderId}`);

    const response = await axios.delete(`${API_URL}/reminders/${reminderId}`, {
      headers: {
        Authorization: `Bearer ${token}`,
      },
    });

    console.log("Reminder Deleted:", response.data);
    return response.data;
  } catch (error) {
    console.error("Delete Reminder Error:", error.response?.data || "No Response");
    throw error.response?.data?.error || "Failed to delete reminder";
  }
};
```

Figure 22: Method in petService.js that sends a POST | GET | DELETE request with pet reminders to the backend

Backend (Reminders): The `/api/reminders/<pet_id>` endpoint receives a GET request to retrieve all reminders associated with a specific pet by querying the reminders collection in MongoDB using the provided `pet_id`. The `/api/reminders/owner/<owner_id>` endpoint handles a GET request to fetch all reminders linked to a particular owner by searching the collection with the `owner_id`. To create a new reminder, the `/api/reminders` endpoint accepts a POST request, validates the incoming data, and saves the reminder to the database, associating it with the specified `pet_id` and `owner_id`. Lastly, the `/api/reminders/<reminder_id>` endpoint handles a DELETE request to remove a reminder, locating and deleting the corresponding document from the collection using the provided `reminder_id`.

```
@app.route(f"{API_PREFIX}/reminders", methods=["POST"])
def add_reminder():
    data = request.json

    # Validate input
    if "owner_id" not in data or "pet_id" not in data or "date" not in data:
        print("Missing required fields", data)
        return jsonify({"error": "Missing required fields (owner_id, pet_id, date)"}), 400

    try:
        owner_obj_id = ObjectId(data["owner_id"])
        pet_obj_id = ObjectId(data["pet_id"])
    except Exception as e:
        print("Invalid owner_id or pet_id format", str(e))
        return jsonify({"error": "Invalid owner_id or pet_id format"}), 400

    try:
        # Normalize the reminder type using aliases on top of this code
        user_type = data.get("type", "Other").lower()
        normalized_type = reminder_type_aliases.get(user_type, user_type.title())

        reminder_id = reminders.insert_one({
            "owner_id": owner_obj_id,
            "pet_id": pet_obj_id,
            "date": data["date"], # (format: YYYY-MM-DD)
            "type": normalized_type,
            "notes": data.get("notes", ""),
            "status": "Pending"
        }).inserted_id

        print("Reminder added successfully", reminder_id)

        # Update the pet's lastReminders field for the proactive reminder check
        pets.update_one(
            {"_id": pet_obj_id},
            {"$set": {f"lastReminders.{normalized_type}": data["date"]}}
        )

        return jsonify({"message": "Reminder added successfully", "reminder_id": str(reminder_id)}), 201

    except Exception as e:
        print("Error adding reminder", str(e))
        return jsonify({"error": "Error adding reminder"}), 500
```

```
# -----
# API: Get Reminders for a pet
# -----
@app.route(f"{API_PREFIX}/reminders/<pet_id>", methods=["GET"])
def get_reminders(pet_id):
    try:
        pet_obj_id = ObjectId(pet_id) # Validate pet_id format
    except Exception:
        return jsonify({"error": "Invalid pet_id format"}), 400

    reminders_list = list(reminders.find({"pet_id": pet_obj_id,
                                          {"_id": 1, "date": 1, "type": 1, "notes": 1, "status": 1, "pet_id": 1}}))

    if not reminders_list:
        return jsonify({"message": "No reminders found for this pet"}), 404

    # Convert ObjectId to string before returning the response
    for reminder in reminders_list:
        reminder["_id"] = str(reminder["_id"])
        reminder["pet_id"] = str(reminder.get("pet_id", "unknown"))

    return jsonify(reminders_list), 200
```

```
# -----
# API: Get ALL Reminders for an owner
# -----
@app.route(f"{API_PREFIX}/reminders/owner/<owner_id>", methods=["GET"])
def get_all_reminders(owner_id):
    try:
        owner_obj_id = ObjectId(owner_id) # Validate owner_id format
    except Exception:
        return jsonify({"error": "Invalid owner_id format"}), 400

    reminders_list = list(reminders.find({"owner_id": owner_obj_id,
                                          {"_id": 1, "date": 1, "type": 1, "notes": 1, "status": 1, "pet_id": 1}}))

    if not reminders_list:
        return jsonify({"message": "No reminders found for this owner"}), 404

    # Convert ObjectId to string before returning the response
    for reminder in reminders_list:
        reminder["_id"] = str(reminder["_id"])
        reminder["pet_id"] = str(reminder.get("pet_id", "unknown"))

    return jsonify(reminders_list), 200
```

```
# -----
# API: Delete a Reminder
# -----
@app.route(f"{API_PREFIX}/reminders/<reminder_id>", methods=["DELETE"])
def delete_reminder(reminder_id):
    try:
        reminder_obj_id = ObjectId(reminder_id) # Validate reminder_id format
    except Exception:
        return jsonify({"error": "Invalid reminder_id format"}), 400

    # Check if reminder exists
    reminder = reminders.find_one({"_id": reminder_obj_id})
    if not reminder:
        return jsonify({"error": "Reminder not found"}), 404

    pet_id = reminder["pet_id"] # Get the pet_id from the reminder
    reminder_type = reminder["type"] # Get the type of the reminder

    # Update the pets lastReminders field to remove the deleted reminder
    result = reminders.delete_one({"_id": reminder_obj_id})

    if result.deleted_count == 0:
        return jsonify({"error": "Reminder not found"}), 404

    # Check if there are other reminders of this type for this pet
    latest_reminder = reminders.find_one(
        {"pet_id": pet_id, "type": reminder_type},
        sort=[("date", -1)] # Get the date of most recent remaining reminder
    )

    if latest_reminder:
        # Update lastReminders with the new most recent date
        pets.update_one(
            {"_id": pet_id},
            {"$set": {f"lastReminders.{reminder_type}": latest_reminder["date"]}}
        )
    else:
        # No more reminders of this type - remove the field
        pets.update_one(
            {"_id": pet_id},
            {"$unset": {f"lastReminders.{reminder_type}": ""}}
        )

    return jsonify({"message": "Reminder deleted successfully"}), 200
```

Figure 23: Flask API route for reminders that handles requests like POST/GET/DELETE

Database: reminders collection from MongoDB

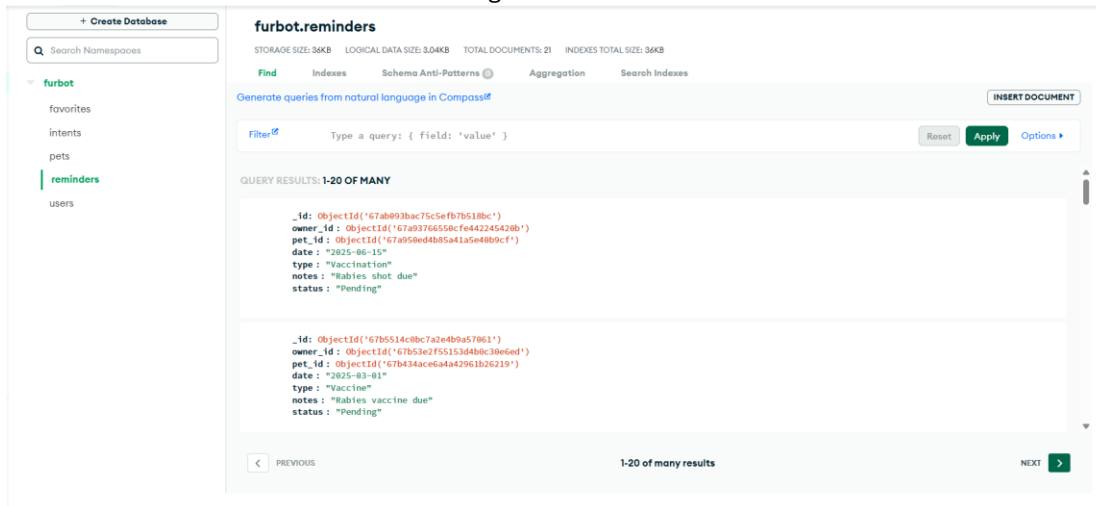


Figure 24: MongoDB reminders collection

Summary of Logic Flow:

Add new reminder:

1. User fills out the reminder form in ReminderList.vue (type, date, and optional note).
2. addNewReminder() is triggered calls addReminder() from the service layer.
3. addReminder() sends a POST request to /api/reminders with the form data and associated pet_id and owner_id.
4. Flask validates the data and inserts the reminder into the reminders collection in MongoDB.
5. The reminder list is refreshed by calling fetchReminders() and updated on the frontend.

To retrieve reminders for a pet:

6. In PetProfile.vue, fetchReminders() is triggered calls getRemindersForPet() from the service layer.
7. getRemindersForPet() sends a GET request to /api/reminders/<pet_id>.
8. Flask fetches all reminders associated with the given pet_id and returns them.
9. The frontend displays the fetched pet-specific reminders.

To retrieve reminders for an owner:

10. When Dashboard.vue is loaded, the mounted() lifecycle hook in ReminderList.vue is triggered.
11. getRemindersForOwner() is called sends a GET request to /api/reminders/owner/<owner_id>.
12. Flask retrieves all reminders associated with the authenticated owner_id and sends them back.

13. The reminders are displayed dynamically in the ReminderList.vue component.

To delete a reminder:

14. User clicks the "Delete" button next to a reminder card in ReminderList.vue.
15. deleteReminder() is called sends a DELETE request to
/api/reminders/<reminder_id>.
16. Flask locates and deletes the reminder from the database.
17. The reminder list is refreshed by calling fetchReminders() again and updating the view.

Implemented Feature 6: Chatbot

Description:

The FurBot Chatbot acts as a virtual pet assistant embedded within the dashboard. It allows users to ask questions, get location-based recommendations (e.g., nearby groomers, vets, pet-friendly cafes), and even save favorite businesses. The chatbot enhances user experience by providing real-time, conversational support powered by Flask and Yelp API.

Frontend (Vue.js): Chatbot.vue

The chatbot UI includes:

- A message input field
- A scrolling message list
- A typing animation while waiting for a response
- Yelp search results (e.g., business cards).

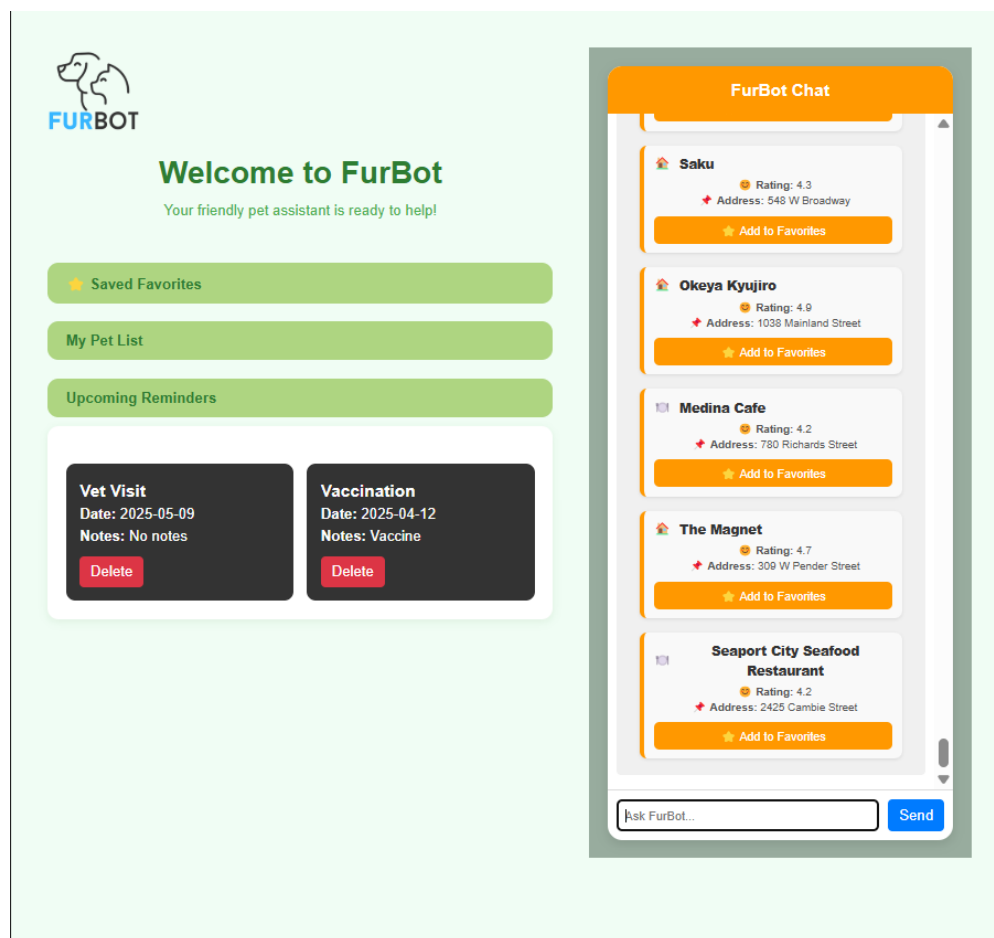


Figure 25: Chatbot UI embedded in Dashboard.vue

Code Highlight: The component Chatbot.vue is embedded in the Dashboard and loads automatically when the user logs in. It provides a real-time chat interface for user interaction.

When the user sends a message, the sendMessageToChatbot() method from chatService.js is called. This method determines whether to:

- Route the message to the /api/chatbot endpoint (to query from the intent in MongoDB), or
- Trigger the Yelp API

Service Layer: Keyword detection logic used to identify if it is a Yelp-related request from service layer.

1. Receives the user message

```
const lowerMessage = message.toLowerCase();
```

2. Scans the message for service-related keywords

```
// Pet Service Detection
if(
  lowerMessage.includes("find") ||
  lowerMessage.includes("search") ||
  lowerMessage.includes("nearest") ||
  lowerMessage.includes("near") ||
  lowerMessage.includes("nearby") ||
  lowerMessage.includes("near me")
) {
```

3. If matched, it triggers the Yelp-related logic.
 - Searches for keywords like groomer, vet, restaurant, hotel, etc.
 - Determines what kind of pet service to search for (i.e., sets searchTerm, category, and attributes)
 - Defaulted the location to Vancouver, BC if no location found.

```

if(lowerMessage.includes("groomer") || lowerMessage.includes("grooming")) {
  searchTerm = "pet groomers";
  category = "groomer";
}
else if(lowerMessage.includes("vet") || lowerMessage.includes("veterinarian")) {
  searchTerm = "veterinarians";
  category = "vet";
}
else if(lowerMessage.includes("pet store") || lowerMessage.includes("shop")) {
  searchTerm = "pet store";
  category = "petstore";
}
else if(lowerMessage.includes("dog park") || lowerMessage.includes("park")) {
  searchTerm = "dog park";
  category = "dog_parks";
}
else if(lowerMessage.includes("daycare") || lowerMessage.includes("boarding")) {
  searchTerm = "pet daycare";
  category = "petboarding";
}
else if(lowerMessage.includes("trainer") || lowerMessage.includes("training")) {
  searchTerm = "dog training";
  category = "pet_training";
}
else if(lowerMessage.includes("restaurant") || lowerMessage.includes("cafe") || lowerMessage.includes("eat")) {
  searchTerm = "restaurants";
  category = "restaurants";
  attributes = "dog_friendly";
}
else if(lowerMessage.includes("hotel") || lowerMessage.includes("stay") || lowerMessage.includes("accommodation")) {
  searchTerm = "hotels";
  category = "hotels";
  attributes = "dog_friendly";
}
else {
  searchTerm = "pet services";
  category = "petservices,groomer,vet,dog_parks,petstore";
}

const location = "Vancouver, BC"; // Default Location

```

4. Sends GET request to /api/yelp/search and format the Yelp pet service result.

```

// Call Yelp API for Pet Services
const response = await axios.get(`${API_URL}/yelp/search`, {
  params:{term: searchTerm, location: location, category: category}
});

console.log("Yelp full response:", response.data);

if(!response.data || !response.data.businesses || response.data.businesses.length === 0) {
  return{reply: `I couldn't find any ${searchTerm} near you. Try a different location!`};
}

// Format Pet Service Results
const services = response.data.businesses.map(service => ({
  name: service.name,
  rating: service.rating || "N/A",
  address: service.location?.address1 || "No address available"
})));

return {reply: services};

```

5. If the message does not contain a service/location keywords, it then send POST request to /api/chatbot endpoint and check the MongoDB intent collection and return the response saved from the collection.

```
// Default Chatbot Response
const chatbotResponse = await axios.post(`${API_URL}/chatbot`, {message});

return chatbotResponse.data && chatbotResponse.data.reply
  ? chatbotResponse.data
  : {reply: "I don't understand that. Please try again."};
```

Figure 26: Different parts of chatService.js

Backend: The /api/chatbot endpoint handles a POST request containing the user's message. Upon receiving the message, the server performs intent detection by analyzing the content and matching it against predefined intents stored in a MongoDB collection. If a matching intent is found, the server responds with a corresponding reply. If no match exists, a fallback response is returned (e.g., "I don't understand that").

1. If the message includes “hello”, “hi” or “help”, a static answer returned.

```
if "hello" in user_message_lower or "hi" in user_message_lower:
    return jsonify({"reply": "🐾 Hello! I'm FurBot. Ask me anything about pet travel, care, or services!"})
elif "help" in user_message_lower:
    return jsonify({"reply": "💡 Try asking things like:\n• Where can I find a groomer?\n• How do I fly with my dog?\n• Show pet stores in Vancouver"})
```

2. Using the spaCy NLP model in Python it attempts to extract GPE (Geopolitical Entity) entities such as city names from the user's message. If no GPE is detected by spaCy, it falls back to a regular expression to extract the location from the message. If both methods fail, it defaults to "Vancouver, BC".

```
# Run spaCy NLP to detect location entities
doc = nlp(user_message.title()) # Capitalizing helps spaCy detect proper nouns
gpe = next((ent.text for ent in doc.ents if ent.label_ == "GPE"), None) # Get the GPE based on the user message

# If no GPE detected, use regex to find location in the message
if not gpe:
    match = re.search(r"in ([A-Z][a-z]+(?: [A-Z][a-z]+)?)", user_message)
    if match:
        gpe = match.group(1)

detected_location = gpe or "Vancouver, BC" # Default to Vancouver if no Location detected
```

3. Load intent documents from intents collection. Using the the fuzzyWuzzy (fuzz.token_set_ratio) the system compares the message against all intents from the MongoDB, the highest matching intent is selected if it scores 60 or above.

```
# Fuzzywuzzy to match user message against intent examples
for intent in intents:
    for example in intent.get("examples", []):
        # Compare input with each example using fuzzy string matching
        score = fuzz.token_set_ratio(user_message_lower, example.lower())
        print(f"Comparing with: {example} | Score: {score}")
        if score > best_score:
            best_score = score
            best_match = intent

# Return best match if confidence is high enough
if best_match and best_score >= 60:
    intent_name = best_match["intent"]
    print(f"Matched Intent: {intent_name} | Confidence: {best_score}")
```

4. The best match checks if it is in intent_to_yelp_config it then calls search_yelp() from yelp_service.py

```
# Yelp intent configuration
intent_to_yelp_config = {
    "local_vet_search": {"term": "veterinarians", "category": "vet"},
    "grooming_request": {"category": "groomer"},
    "pet_friendly_hotels": {"category": "hotels", "attributes": "dog_friendly"},
    "pet_friendly_restaurants": {"category": "restaurants", "attributes": "dog_friendly"},
    "dog_parks_nearby": {"category": "dog_parks"},
    "pet_hiking_spots": {"category": "hiking"},
    "pet_stores_nearby": {"term": "pet store", "category": "petstore"},
    "pet_friendly_malls": {"term": "shopping", "category": "shopping", "attributes": "dog_friendly"},
    "near_me_search": {"term": "pet services"},
    "local_services_city": {"term": "pet groomers"},
    "find_pet_friendly_groomers": {"category": "groomer", "attributes": "dog_friendly"},
    "find_pet_friendly_vets": {"category": "vet", "attributes": "dog_friendly"},
    "find_pet_friendly_stores": {"category": "petstore", "attributes": "dog_friendly"},
    "find_pet_friendly_cafes": {"category": "cafes", "attributes": "dog_friendly"}
}
```

```
if intent_name in intent_to_yelp_config:
    yelp_config = intent_to_yelp_config[intent_name]
    print("Yelp Config:", yelp_config)

    yelp_result = search_yelp(
        term=yelp_config.get("term", "pet services"),
        location=detected_location,
        category=yelp_config.get("category"),
        attributes=yelp_config.get("attributes")
    )

    businesses = yelp_result.get("businesses", [])
    results = [
        {
            "name": b.get("name", "No name"),
            "rating": b.get("rating", "N/A"),
            "address": b.get("location", {}).get("address1", "No address"),
            "image_url": b.get("image_url", ""),
            "id": b.get("id", b.get("name", "unknown"))
        } for b in businesses
    ]

    return jsonify({
        "reply": results,
        "intent": intent_name,
        "confidence": best_score,
        "location_detected": detected_location
    })
```

5. Search_yelp function from the yelp_service.py

```
def search_yelp(term="pet services", location="Vancouver, BC", category=None, attributes=None):
    headers = {
        "Authorization": f"Bearer {API_KEY}",
        "Content-Type": "application/json"
    }
```

Figure 27: Different parts of api/chatbot and /yelp/search

Database: intents collection from MongoDB

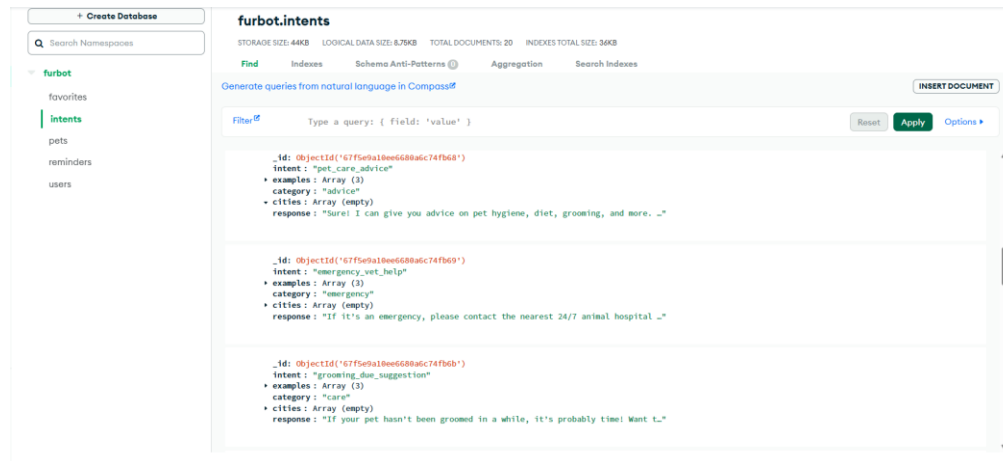


Figure 28: intents collection from MongoDB

Summary of Logic Flow:

Send user message to chatbot:

1. User types a message in the chatbot input field in Chatbot.vue.
2. Then sendMessage() method is triggered calls sendMessageToChatbot() from chatService.js.
3. sendMessageToChatbot() checks if the message contains keywords like “find” or “near” to determine if the request is location-based.
4. If location-based, it uses spaCy NLP and fallback regex to detect city or region in the message.
5. Once the location and service type (e.g., “groomer”, “vet”) are determined, a Yelp API request is sent to retrieve matching pet services.
6. The API response is returned to the frontend and displayed in the chat window as service cards (e.g., name, address, contact info).

For general chatbot questions (non-location-based):

7. If no location-based keywords are detected, the message is sent via a POST request to the /api/chatbot endpoint.
8. Flask receives the message and matches it against stored intents in the MongoDB intents collection using fuzzy matching.
9. A response is generated (either static or calls the search_yelp function from yelp_service.py) and returned to the frontend.
10. The chatbot displays the reply in the chat window.

Implemented Feature 7: Saving to Favorites

Description:

This feature allows users to save pet-related businesses (e.g., vets, groomers, pet stores) to their Favorites list directly from the chatbot. When the chatbot shows a list of businesses (via Yelp), users can click **"Add to Favorites"**. The selected business is saved in MongoDB and displayed on the dashboard under the **Saved Favorites** section.

Frontend (Vue.js):

Chatbot.vue

- Renders business cards with an "Add to Favorites" button.
- When clicked, it calls the `saveToFavorites()` method from `Chatbot.vue`.
- Emits the `@favorite-added` event, which notifies `Dashboard.vue` to refresh the favorite list.

Dashboard.vue

- Displays the `<FavoriteList />` component inside a collapsible card.
- Listens for the event `@favorite-added` from `Chatbot.vue` and refreshes the favorites.

FavoriteList.vue

- Fetches the current user's favorites on mount.
- Displays each saved business with name, address, rating, and delete button.

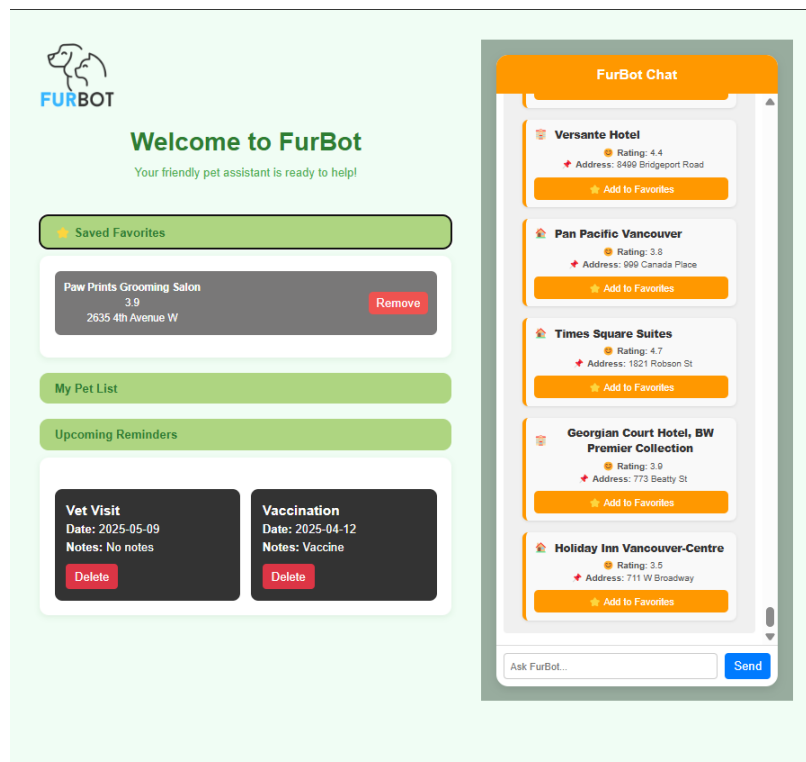


Figure 29: Chatbot UI with Add to Favorites

Code Highlight: Inside Chatbot.vue, when a user clicks "Add to Favorites" on a business card, the component calls the saveToFavorites() function from the Chatbot.vue that sends the POST request to /api/favorites. It then emits a @favorite-added event, which is captured by the handleFavoriteAdded() method in Dashboard.vue, triggering the FavoriteList.vue component to refresh and display the updated list of saved businesses.

```
handleFavoriteAdded() {
  const favListRef = this.$refs.favoriteList;
  if (favListRef && typeof favListRef.fetchFavorites === 'function') {
    favListRef.fetchFavorites();
  }
},

async saveToFavorites(business) {
  const ownerId = localStorage.getItem("user_id");
  if (!ownerId) {
    alert("You need to be logged in to save favorites.");
    return;
  }

  const payload = {
    owner_id: ownerId,
    business_id: business.business_id || business.name,
    name: business.name || "Unnamed",
    image_url: business.image_url || "",
    rating: business.rating || "N/A",
    location: business.address || "Unknown location",
  };

  try {
    const response = await fetch("/api/favorites", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(payload),
    });

    const result = await response.json();

    if (response.ok) {
      alert("✅ Added to Favorites!");
      this.$emit("favorite-added");
    } else {
      alert(result.error || "Something went wrong.");
    }
  } catch (err) {
    console.error("Failed to save favorite:", err);
    alert("Failed to save favorite.");
  }
}
```

Figure 30: Service Layer for Favorites

Backend: The service layer provides functions to manage user favorites: `saveToFavorites` (business) sends a **POST** request to `/api/favorites` with the owner's ID and selected business data to save it as a favorite, including an authorization token in the header. `getFavorites(ownerId)` sends a **GET** request to retrieve all saved businesses associated with the given owner. `deleteFavorite(favoriteId)` sends a **DELETE** request to `/api/favorites/<favoriteId>` to remove a specific favorite from the database.

```
# -----
# API: Saving a favorite result to favorites
# -----
@app.route(f"{API_PREFIX}/favorites", methods=["POST"])
def add_favorite():
    data = request.json

    try:
        # to prevent duplicate entries, check if the user already has this favorite
        existing = favorites.find_one({
            "owner_id": ObjectId(data["owner_id"]),
            "business_id": data["business_id"]
        })
        if existing:
            return jsonify({"error": "This is already in your favorites."}), 200

        data["owner_id"] = ObjectId(data["owner_id"]) # Convert owner_id to ObjectId
        favorites.insert_one(data) # Insert the favorite into the database
        return jsonify({"message": "Favorite added successfully!"}), 201
    except Exception as e:
        print("Error adding favorite:", str(e))
        return jsonify({"error": "Failed to add favorite"}), 500

# -----
# API: Get all favorites for a user
# -----
@app.route(f"{API_PREFIX}/favorites/owner_id", methods=["GET"])
def get_favorites(owner_id):
    try:
        owner_obj_id = ObjectId(owner_id) # Validate owner_id format
    except Exception:
        return jsonify([]), 200 # Return empty list if invalid owner_id

    favorites_list = list(favorites.find({"owner_id": owner_obj_id, "_id": 1, "business_id": 1, "owner_id": 1, "name": 1, "image_url": 1, "rating": 1, "location": 1})) # Get all favorites for the user

    # Convert ObjectId to string before returning the response
    for favorite in favorites_list:
        favorite["_id"] = str(favorite["_id"])
        favorite["owner_id"] = str(favorite.get("owner_id", "unknown"))

    return jsonify(favorites_list), 200

# -----
# API: Delete a favorite
# -----
@app.route(f"{API_PREFIX}/favorites/<favorite_id>", methods=["DELETE"])
def delete_favorite(favorite_id):
    try:
        favorite_obj_id = ObjectId(favorite_id) # Validate favorite_id format
    except Exception:
        return jsonify({"error": "Invalid favorite_id format"}), 400

    # Check if favorite exists
    favorite = favorites.find_one({"_id": favorite_obj_id})
    if not favorite:
        return jsonify({"error": "Favorite not found"}), 404

    favorites.delete_one({"_id": favorite_obj_id})
    return jsonify({"message": "Favorite deleted successfully"}), 200
```

Figure 31: Backend AIP endpoints for Favorites

Database: favorites collection from MongoDB

The screenshot shows the MongoDB Compass interface for the 'furbot' database, specifically the 'favorites' collection. The left sidebar shows the database structure with 'favorites' selected. The main area displays the collection's details: STORAGE SIZE: 36KB, LOGICAL DATA SIZE: 575B, TOTAL DOCUMENTS: 3, INDEXES TOTAL SIZE: 36KB. Below this, there are tabs for 'Find', 'Indexes', 'Schema', 'Anti-Patterns', 'Aggregation', and 'Search Indexes'. The 'Find' tab is active, showing a query filter bar with the text 'Type a query: { field: 'value' }'. Below the filter bar, it says 'QUERY RESULTS: 1-3 OF 3'. Three documents are listed, each with the following fields: _id, owner_id, business_id, name, image_url, rating, and location.

```

_id: ObjectId('67f0dac158b27d1f964f0de1')
owner_id: ObjectId('67e7a1f6943842ab099c800a')
business_id: "Paw Prints Grooming Salon"
name: "Paw Prints Grooming Salon"
image_url: ""
rating: 3.9
location: "2635 4th Avenue W"

_id: ObjectId('67f16570c3bf91eeae2f4ff8')
owner_id: ObjectId('67b03b5c48eb086e4bb1b889')
business_id: "Town & Country Mobile Pet Grooming"
name: "Town & Country Mobile Pet Grooming"
image_url: ""
rating: 5
location: "2416 Western Avenue"

_id: ObjectId('67f4480ff13c2858358f4f71')
owner_id: ObjectId('67f439a4759ed961cf62eb1')
business_id: "South Arm Dog Off Leash Area"

```

Summary of Logic Flow:

Add a favorite service:

1. User clicks the "Add to Favorites" button on a service card displayed by the chatbot.
2. This triggers the saveToFavorites() method in Chatbot.vue, passing the business_id and the logged-in owner_id.
3. A POST request is sent to the /api/favorites endpoint.
4. Flask validates the request and inserts the favorite into the favorites collection in MongoDB, associating it with the user's owner_id.
5. A success message is returned, and the frontend updates accordingly.

Retrieve user favorites:

6. In Dashboard.vue, when the Favorites section is expanded, the FavoriteList.vue component is rendered.
7. The fetchFavorites() method is triggered calls getFavorites() from the service layer.
8. getFavorites() sends a GET request to /api/favorites/<owner_id>.
9. Flask queries the favorites collection for all entries matching the given owner_id.
10. The retrieved list of saved services is returned and displayed as cards in the FavoriteList.vue component.

Delete a favorite service:

11. User clicks the "Delete" icon next to a saved service in the FavoriteList.vue component.
12. This triggers the deleteFavorite() method from the service layer, passing the favorite_id of the selected item.
13. A DELETE request is sent to /api/favorites/<favorite_id>.

14. Flask locates the corresponding entry in the favorites collection and deletes it.
15. After successful deletion, the frontend calls fetchFavorites() again to refresh and update the list of saved favorites in real time.

Implemented Feature 8: Automated Email Reminder

Description:

This feature automatically scans upcoming pet care reminders (e.g., grooming, vaccinations, vet visits) and sends an email notification to the pet owner. It uses a scheduled background task powered by APScheduler to check reminders on a daily basis. When a relevant reminder is found an email is composed and sent using Flask-Mail. Additionally, it proactively/smarts checks the if the last reminder for grooming is above 30 days and vaccination 365 days, it will remind the user to schedule a reminder.

Backend: A scheduled task (check_reminders) runs daily and scans the reminders collection for any appointments scheduled for the current day (e.g., grooming, vaccinations). If a match is found, the system sends a personalized reminder email to the pet owner and updates the status to "Completed" to prevent duplicate notifications.

In addition to daily alerts, FurBot also features a proactive reminder system (check_proactive_reminders) that checks for overdue care routines based on each pet's history. Specifically, if a pet hasn't received a grooming reminder in the last 30 days or a vaccination in the last 365 days, the system automatically generates a gentle suggestion and sends it via email to the owner. These proactive reminders are stored under each pet's record to ensure suggestions are not repeatedly sent unless necessary.

1. Scheduler Setup

```
# Schedule the job to run
scheduler.add_job(id="check_reminders", func=check_reminders, trigger="cron", minute="*") # For testing, run every minute

scheduler.add_job(id="check_proactive_reminders", func=check_proactive_reminders, trigger="cron", minute="*") # For testing, run every minute change to
hour="1" in production to run at 1 AM.
```

2. Reminder Matching Logic

```
def check_reminders():
    vancouver_tz = pytz.timezone("America/Vancouver") # Set timezone to Vancouver
    # Set the date to the current date in Vancouver timezone
    today = datetime.datetime.now(vancouver_tz).date()
    print("Running check_reminders for:", today) # Console log for debugging

    # Fetch reminders for today
    reminders_list = list(reminders.find({"date": str(today), "status": "Pending"}))
    print("Reminders found:", len(reminders_list)) # Console log for debugging

    # If no reminders found, log and return
    if not reminders_list:
        logging.info("No reminders for today")
        return
```

3. Proactive check threshold

```
def check_proactive_reminders():  
    vancouver_tz = pytz.timezone("America/Vancouver") # Set timezone to Vancouver  
    today = datetime.datetime.now(vancouver_tz).date() # Set the date to the current date in Vancouver timezone  
    print("Running check_proactive_reminders for:", today) # Console log for debugging  
  
    reminder_gap_days = {  
        "Grooming": 30, # Grooming reminders every 30 days  
        "Vaccination": 365, # Vaccination reminders every 365 days  
    }
```

4. Suggestion Email Formatting

```
message = (  
    f"It's been over {max_days} days since your last {reminder_type.lower()} for {pet_name}. "  
    f"Log in to FurBot to schedule a {reminder_type.lower()} appointment: https://furbot.app/login"  
)  
suggested.append({"type": reminder_type, "message": message})
```

Evaluation and Techniques

To gather user insights on my MVP product, I utilized a survey-based evaluation technique. Users were invited to log in to the deployed production version of the application via <https://furbot-production.up.railway.app> and subsequently complete a feedback form available at <https://forms.office.com/r/EanTkxbsZ2>. The following section presents an analysis of the responses to each survey question, along with the corresponding action items identified for improvement.

1. How easy was it to navigate and interact with FurBot?

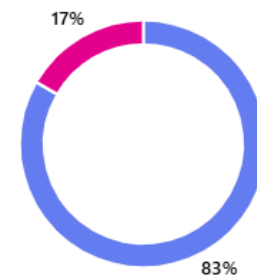
Extremely easy	6
Somewhat easy	0
Neutral	0
Somewhat not easy	0
Extremely not easy	0



Analysis: This clearly states that based on user feedback that FurBot is easy to use.

2. How helpful was FurBot in answering your pet-related questions (e.g., grooming, travel, reminders)?

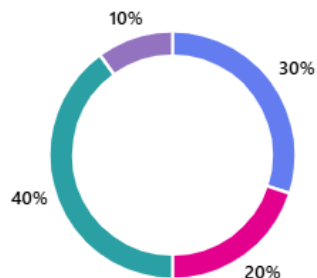
Very helpful	5
Somewhat helpful	1
Neither helpful nor unhelpful	0
Somewhat unhelpful	0
Very unhelpful	0



Analysis: Generally, FurBot is very helpful specially for pet parents.

3. Which FurBot feature did you find most useful? *(Select one or more)*

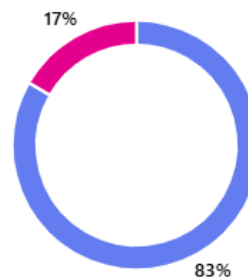
Pet Profile Management	3
Reminder System	2
Chatbot Assistance	4
Email Notifications	1
Other	0



Analysis: The majority of users found the Chatbot Assistance feature most useful, followed by Pet Profile Management and the Reminder System.

4. How likely are you to recommend FurBot to other pet owners?

Very likely	5
Somewhat likely	1
Neither likely nor unlikely	0
Somewhat unlikely	0
Very unlikely	0



Analysis: 83% of users indicated they are **very likely** to recommend FurBot. This reflects strong user satisfaction and suggests a positive overall reception of the MVP.

5. What suggestions do you have to improve FurBot?

6 Responses

ID ↑	Name	Responses
1	anonymous	Good start doing this and excited for the next iteration! ;-)
2	anonymous	Maybe like Common FAQs
3	anonymous	Ability to cater to user's location to check for nearby parks for example and other location-specific inquiries
4	anonymous	Change themes or background, that would be so cute!
5	anonymous	To easily identify which pet each reminder is for, display the pet's name under Upcoming Reminders.
6	anonymous	Overall it's a good application for pet owners. One recommendation would be to add placeholder text or field labels in the 'Add a New Pet' section, as it currently only displays empty input boxes without indicating what information is required.

Analysis: Several users suggested improvements such as adding location-based searches, FAQs, themed UI options, and clearer labels for form fields. In response, spaCy NLP was implemented to support location detection for nearby parks and services, enhancing the chatbot's contextual awareness. The other items/suggestion were added to the backlog.

Reflections/Discussions

After 18 years in the IT industry, FurBot marks a significant personal milestone—as it is the first time, I have independently developed a full-stack product from the ground up. The project has been an insightful and transformative journey that tested both my technical and soft skills. One of the most rewarding aspects was seeing how the integration of external APIs like Yelp and the use of NLP via spaCy made the chatbot responses more intelligent and relevant in real-world scenarios. It was especially satisfying to see users engage with the chatbot, pet profile management, and automated reminders, while also providing positive feedback.

One key challenge I faced was in deploying FurBot to Railway especially managing CORS errors and routing configurations between the frontend (Vue) and backend (Flask). Troubleshooting these issues taught me the importance of environment-based configuration and proper API structuring. Additionally, putting myself in a user's shoes and a pet owner itself made me realize how important small details in UI/UX as well as the detailed information that the chatbot can answer also, how can it be personalized getting automated reminders for pet cares.

Throughout this project, I developed a deeper appreciation for essential soft skills such as time management, resilience, critical thinking, problem-solving, and staying composed under pressure. While technical skills can be learned quickly, it's these soft skills that form the foundation for long-term success in any career.

Work Date/Hours logs

Issue Type	Date Worked On	Sprint	Issue key	Summary	Hours Logged	Status
Epic			SCRUM-5	Implementation		Done
Epic			SCRUM-4	ReportsAndDocumentation		Done
Task	13-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-60	Presentation Preparation	5	Done
Task	12-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-70	Final Report Documentation 5	10	Done
Task	11-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-69	Final Report Documentation 4	5	Done
Task	10-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-68	Final Report Documentation 3	3	Done
Task	09-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-57	Final Report Documentation 2	2	Done
Task	08-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-67	spaCy Implementation for location extraction	5	Done
Task	07-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-66	User Evaluation	2	Done
Task	07-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-64	Personalize Feature for additional novelty Day 4	2	Done
Task	06-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-65	Progress Report #5	1.5	Done
Task	04-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-63	Personalize Feature for additional novelty Day 3	2.5	Done
Task	03-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-62	Personalize Feature for additional novelty Day 2	2.5	Done
Task	02-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-56	Final Report Documentation 1	1	Done
Task	01-04-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-61	Personalize Feature for additional novelty Day 1	3	Done
Task	30-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-54	Progress Report #4	1	Done
Task	29-03-2025 0:00	Sprint 8 - UI and Final Report	SCRUM-59	Title and Abstract	1	Done
Task	28-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-53	Retry Production Environment Deployment	6	Done
Task	26-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-58	UI Enhancement 2	1.5	Done
Task	26-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-29	Store Frequently Asked Questions in MongoDB	3	Done
Task	25-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-55	UI Enhancement 1	4	Done
Task	24-03-2025 0:00	Sprint 7 - Auto Email & UI	SCRUM-52	Automate Email Reminders	2	Done
Task	18-03-2025 0:00	Sprint 6 - Prod Deployment	SCRUM-49	Deploy FurBot to Railway 3	6	Done
Task	17-03-2025 0:00	Sprint 6 - Prod Deployment	SCRUM-48	Deploy FurBot to Railway 2	4	Done
Task	12-03-2025 0:00	Sprint 6 - Prod Deployment	SCRUM-47	Test and Debug - March 12	4	Done
Task	12-03-2025 0:00	Sprint 6 - Prod Deployment	SCRUM-46	Deploy FurBot to Railway 1	4	Done
Task	11-03-2025 0:00	Sprint 6 - Prod Deployment	SCRUM-45	Sign up with Railway	2	Done
Task	10-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-44	Creating Progress Report #2	2	Done
Task	06-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-40	Integrate Yelp with Backend API - March 6	1	Done
Task	06-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-39	Integrate Yelp with Chatbot UI - March 6	1	Done
Task	05-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-38	Integrate Yelp with Backend API - March 5	1	Done
Task	05-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-37	Integrate Yelp with Chatbot UI - March 5	2	Done
Task	05-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-32	Implement Basic Conversation Flow	1	Done
Task	04-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-36	Integrate Yelp with Chatbot UI - March 4	2	Done
Task	04-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-35	Integrate Yelp with Backend API - March 4	2	Done
Task	04-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-30	Create initial Chatbot UI	1	Done
Task	04-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-28	Yelp Registration and Initial Set up	1	Done
Task	03-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-34	Integrate Yelp with Chatbot UI - March 3	1	Done
Task	03-03-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-31	Integrate Yelp with Backend API - March 3	1	Done
Task	26-02-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-27	Implement Basic Chatbot Logic	5	Done
Task	20-02-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-33	Midterm Report and Video	5	Done
Task	19-02-2025 0:00	Sprint 5 - Chatbot Integration	SCRUM-26	Create API Endpoint for Chatbot (/chatbot) and Simple UI Integration	5	Done
Task	18-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-25	UI testing & debugging	6	Done
Task	18-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-23	API integration for authentication & pet management	3	Done
Task	17-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-24	User session handling (store token & authentication state)	4	Done
Task	17-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-22	Basic UI for Adding & Viewing Pets	2	Done
Task	17-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-21	Functional UI for Login & Sign-Up	2	Done
Task	11-02-2025 0:00	Sprint 3 - Backend Setup	SCRUM-20	Set up Job Scheduler to automate the email reminders	3	Done
Task	09-02-2025 0:00	Sprint 3 - Backend Setup	SCRUM-17	Set up Reminders	3	Done
Task	09-02-2025 0:00	Sprint 3 - Backend Setup	SCRUM-16	Create end points for pet profiles	2	Done
Task	09-02-2025 0:00	Sprint 3 - Backend Setup	SCRUM-15	Set up MongoDB for Pet Profiles	1	Done
Task	06-02-2025 0:00	Sprint 4 - Building Initial UI	SCRUM-18	Progress Report #1	2	Done
Task	06-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-14	Test API	1	Done
Task	06-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-12	Build User Authentication API	3	Done
Task	05-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-11	Set Up MongoDB	1	Done
Task	05-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-10	Install Dependencies	1	Done
Task	05-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-9	Set Up Python and Virtual Environment	1	Done
Task	05-02-2025 0:00	Sprint 2 - Set Up the Backend	SCRUM-8	Create the Folder Structure in the Implementation folder	1	Done
Task	25-01-2025 0:00	Sprint 1 - FurBot Setup	SCRUM-1	Draft Project Proposal	6	Done
Task	19-01-2025 0:00	Sprint 1 - FurBot Setup	SCRUM-6	Set Up Project Repository	1	Done
Task	19-01-2025 0:00	Sprint 1 - FurBot Setup	SCRUM-2	Set Up JIRA for Project Management	2	Done
Task	18-01-2025 0:00	Sprint 1 - FurBot Setup	SCRUM-13	Research a topic for the project proposal	2	Done
Task	18-01-2025 0:00	Sprint 1 - FurBot Setup	SCRUM-7	Research and Identify Technologies for FurBot	2	Done
TOTAL					163	

Concluding Remarks

First of all, I would like to sincerely thank Professor Padmapriya Arasanipalai Kandhadai for her continuous support and encouragement throughout this journey. Her guidance consistently pushed me to go further and think more critically.

FurBot began as a simple concept a chatbot to assist pet owners, but it evolved into a fully functional and intelligent web application through iterative development and applied research. This project not only allowed me to apply technical knowledge in real-world contexts such as API integration, natural language processing, and deepened my understanding in full-stack deployment/development and practical problem-solving.

Through continuous feedback from Professor and user evaluation, technical challenges, and continuous testing, FurBot became more than just a school project, it became a demonstration of how thoughtfully implemented technology can enhance everyday life. Moving forward, the experience and lessons gained from this project will serve as a strong foundation for future innovation and my continued growth in the tech industry.

References

YouTube Videos:

- **Corey Schafer. (n.d.). *Flask Tutorials - Profile Management*** [YouTube playlist]. YouTube. <https://www.youtube.com/playlist?list=PL-osiE80TeTs4UjLw5MM6OjgkjFeUxCYH>
Used for research in implementing Flask for user authentication and profile management in my project.
- **Sheryians Coding School. (2022, June 11). *Flask and Chatbot*** [Video]. YouTube. https://www.youtube.com/watch?v=70H_7C0kMbl&t=170s
Used for research in implementing backend (Flask) for chatbot.
- **Tech With Tim. (2019, April 30). *Fuzzywuzzy for chatbot*** [Video]. YouTube. <https://www.youtube.com/watch?v=1jNNde4k9Ng>
Used for research in implementing backend (Flask) for string matching for chatbot.

AI Tools:

- **OpenAI. (2025). *ChatGPT*** [AI language model, paid subscription]. <https://chat.openai.com>
Used extensively as a conversational assistant throughout the project. ChatGPT supported backend and frontend development by offering help with coding, debugging, and logic design. It also contributed to shaping the chatbot's behavior and flow by providing guidance on conversational structure and natural language handling. Beyond development, ChatGPT was instrumental in enhancing the quality and clarity of project reports, offering suggestions for wording, structure, and presentation. Additionally, it played a key role in helping me understand and explore new technologies that I used for the first time in this project, such as Flask-Mail, APScheduler, JWT, Flask and MongoDB.
- **GitHub. (n.d.). *GitHub Copilot*** [AI coding assistant]. Integrated to Visual Studio Code.
Used as a built-in assistant within Visual Studio Code to support both backend and frontend development. GitHub Copilot provided real-time code suggestions, helping accelerate the coding process and improve efficiency. When used alongside ChatGPT, it offered a powerful combination — with

Copilot suggesting code directly within the IDE, and ChatGPT providing deeper explanations, logic validation, and troubleshooting support. This collaboration significantly enhanced my development workflow and confidence in working with unfamiliar technologies.

Other Tools:

- **Yelp Inc. (n.d.). Yelp Fusion API. Yelp.**
<https://www.yelp.com/developers/documentation/v3>
Used for real-time data on nearby pet-related services, such as groomers, veterinarians, and pet stores, based on city names detected in user input.
- **Railway. (n.d.). Modern software deployment made simple.**
<https://railway.app/>.
Used for hosting FurBot, allowing the Flask backend and MongoDB integration to be deployed and accessed in a live environment.
- **Atlassian. (n.d.). JIRA Software.**
<https://www.atlassian.com/software/jira>
Used for project tracking, task management, and logging development hours throughout the FurBot project.
- **GitHub. (n.d.). GitHub.**
<https://github.com>
Used for version control and repository management to track code changes, and maintain the overall project codebase and reports.
- **MongoDB, Inc. (n.d.). MongoDB Atlas [Cloud database service].**
<https://www.mongodb.com/atlas>
Used to store and manage user data, pet profiles, and reminder records for the FurBot system.

Appendix A: Installation Guide

1. Open any modern web browser (e.g., Google Chrome, Microsoft Edge, or Safari).
2. In the address bar, type the following URL and press Enter:
<https://furbot-production.up.railway.app/>

Appendix B: User Guide



FurBot User Guide

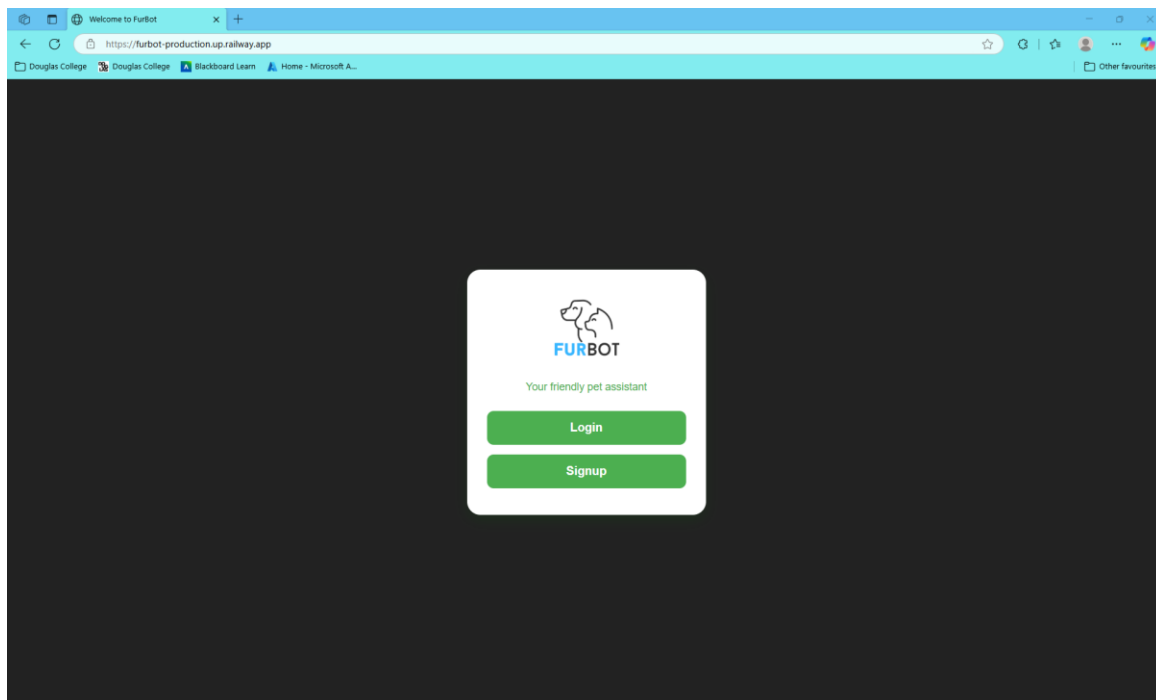
Your smart assistant for managing pet care.

Getting Started

Access FurBot

Visit the deployed application here:

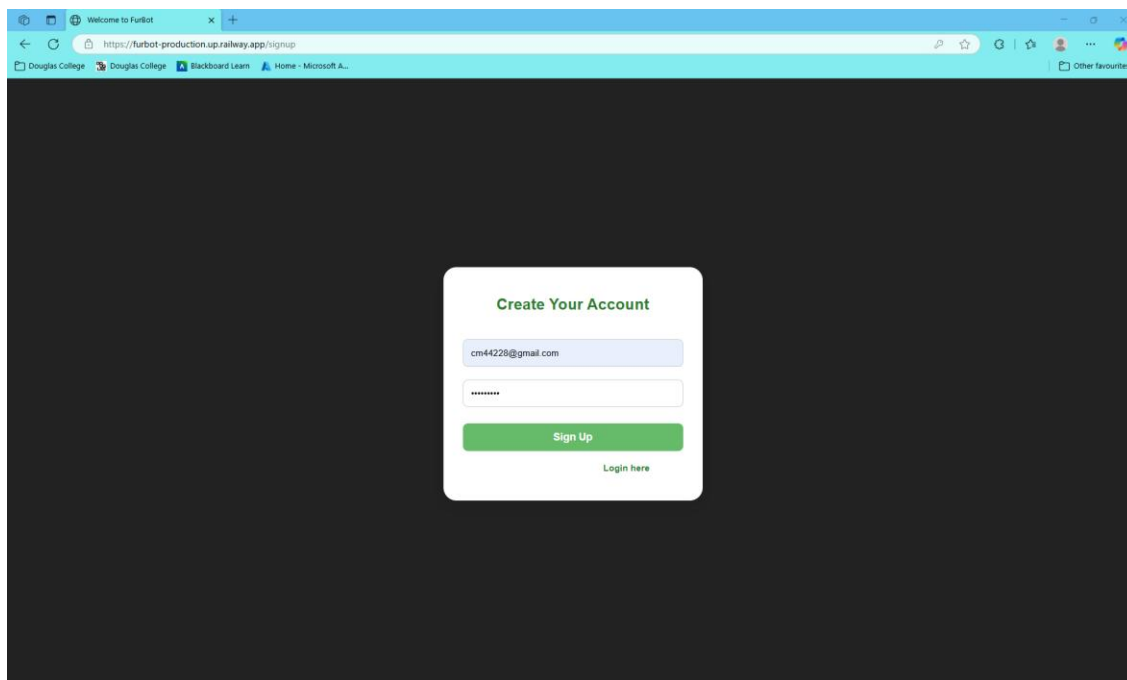
<https://furbot-production.up.railway.app>



Sign Up / Login

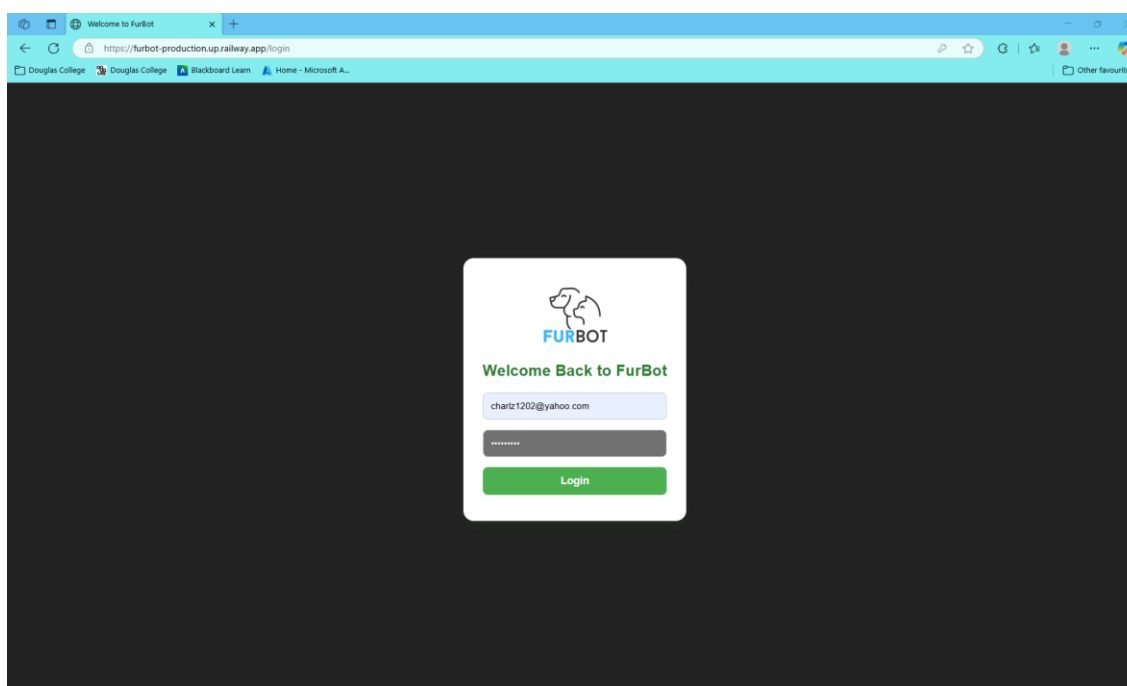
New User Registration:

1. Click "**Signup**" to create an account.
2. Enter a **valid email address** — this is required to receive pet care reminders.
3. Enter a **secure password**.
4. Click "**Signup**" to complete the registration.
5. You will be automatically redirected to the **Login** page.



Returning users: Log in with your credentials:

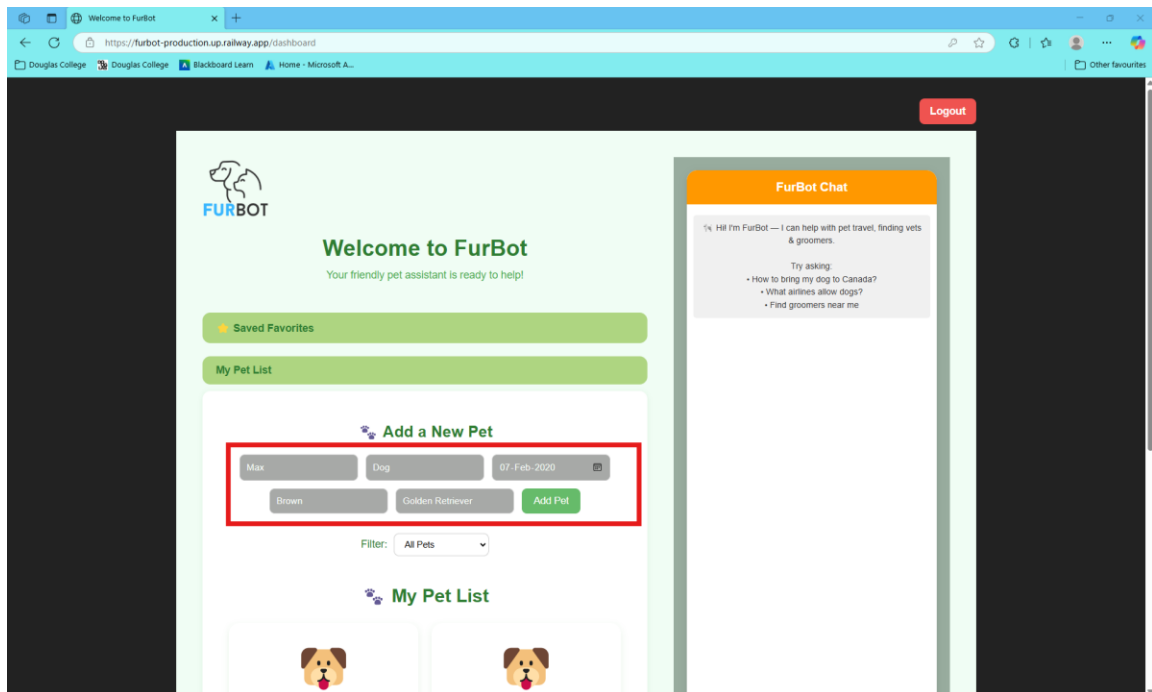
1. Enter an **email address**.
2. Enter a **secure password**.
3. Click "**Login**" to complete the registration.
4. You will be automatically redirected to the **Dashboard** page.



Managing Your Pet Profile

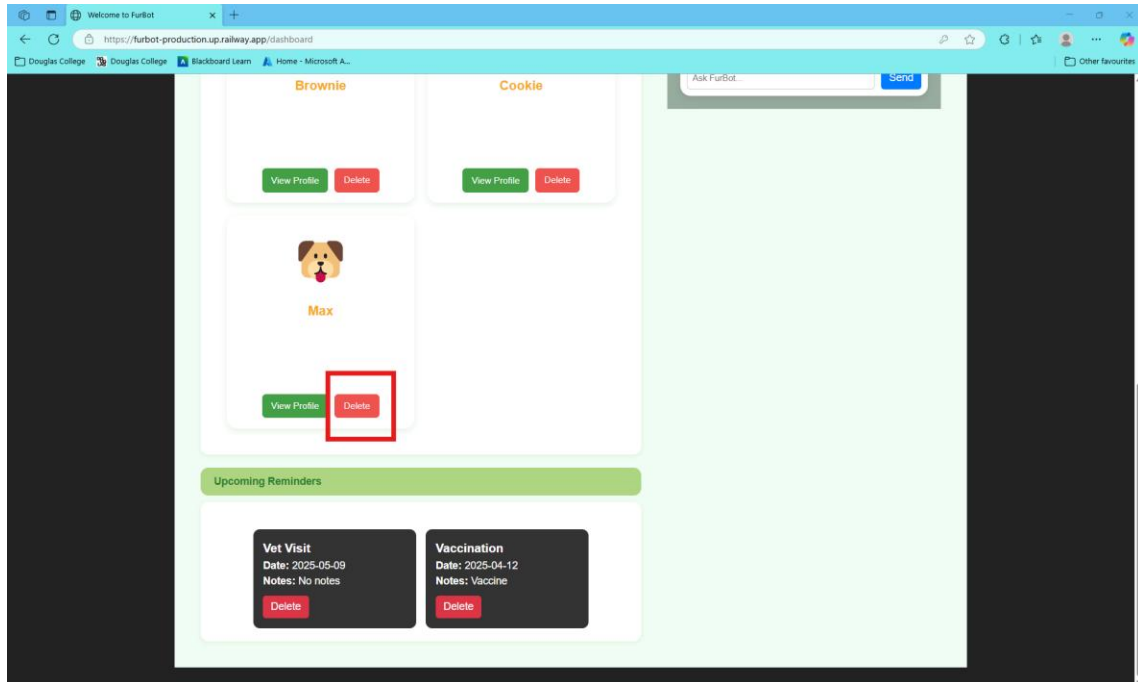
Add a Pet

1. Navigate to the **Dashboard**.
2. Under "**My Pet List**", fill in your pet's details:
 - **Pet Name**
 - **Species** (e.g., Dog, Cat)
 - **Birthdate**
 - **Color**
 - **Breed**
3. Click "**Add Pet**".
4. Your pet will be **automatically added** to the list.
5. An appropriate **icon** (🐕 or 🐈) will be assigned based on the selected species.



Delete a Pet

1. Click the “Delete” button from the pet’s card to remove the pet from the profile.

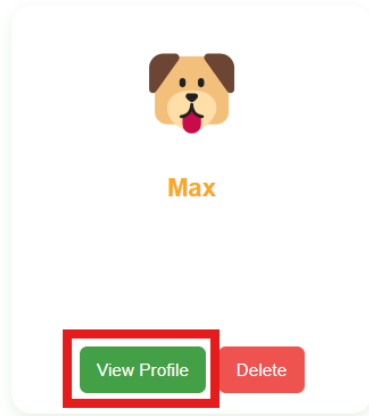


Viewing Pet Profile and Setting a Reminders for a Pet

Adding a Reminder

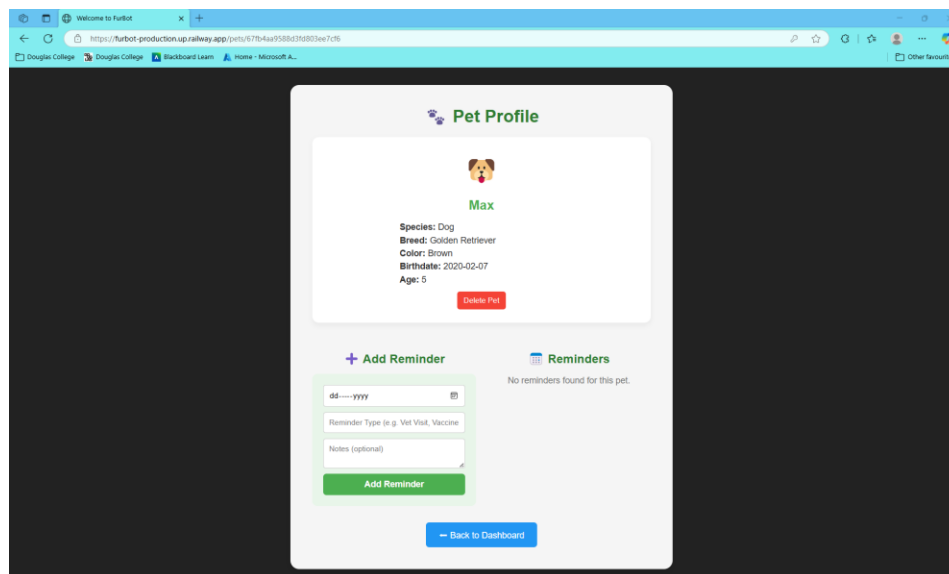
To view your pet's full profile and manage reminders:

1. On the **Dashboard**, under your pet's card, click **"View Profile"**.



2. You will be taken to the **Pet Profile** page, where you can view:

- **Pet Name**
- **Species**
- **Breed**
- **Color**
- **Birthdate**
- **Age**



3. To **add a reminder** for this pet:

- Select the **date** of the reminder.
- Enter the **reminder type** (e.g., Vet Visit, Vaccine).
- Optionally, enter any **notes**.
- Click **“Add Reminder”** to save.

Pet Profile

Max

Species: Dog
Breed: Golden Retriever
Color: Brown
Birthdate: 2020-02-07
Age: 5

[Delete Pet](#)

+ Add Reminder

13-Apr-2025

Grooming

Send Max to grooming

[Add Reminder](#)

Reminders

No reminders found for this pet.

[← Back to Dashboard](#)

4. The reminder will appear in the **Reminders** section on the right.

Pet Profile

Max

Species: Dog
Breed: Golden Retriever
Color: Brown
Birthdate: 2020-02-07
Age: 5

[Delete Pet](#)

+ Add Reminder

dd-....yyyy

Reminder Type (e.g. Vet Visit, Vaccine)

Notes (optional)

[Add Reminder](#)

Reminders

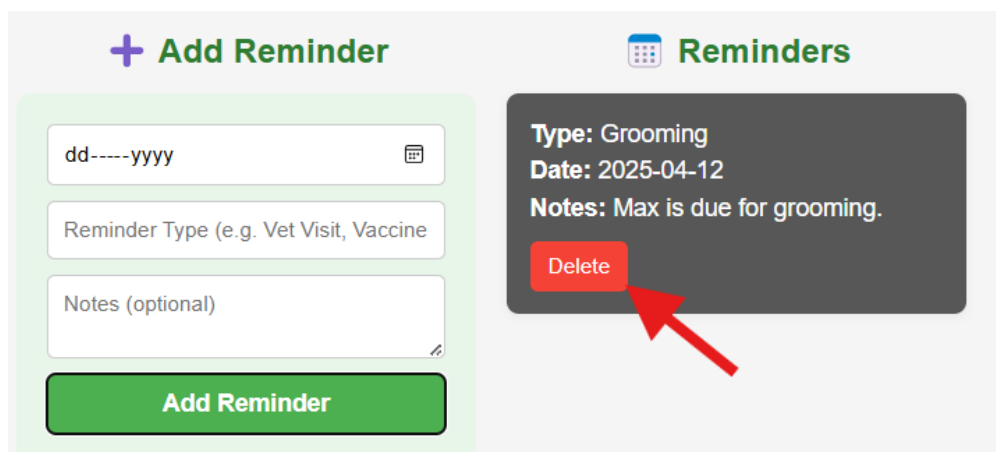
Type: Grooming
Date: 2025-04-13
Notes: Send Max to grooming

[Delete](#)

[← Back to Dashboard](#)

Delete a Reminder

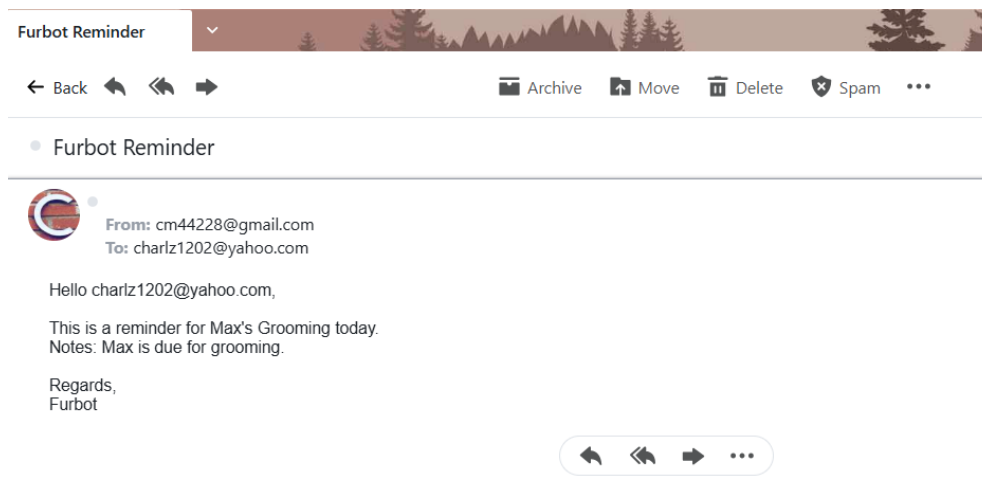
1. Click **"Delete"** next to a reminder to remove it.



The screenshot shows the Furbot Reminders interface. On the left is the 'Add Reminder' form with fields for a date (dd-yyy), a reminder type (e.g., Vet Visit, Vaccine), and optional notes. A green 'Add Reminder' button is at the bottom. On the right is the 'Reminders' section, which displays a list of reminders. A reminder card is shown with the following details: Type: Grooming, Date: 2025-04-12, and Notes: Max is due for grooming. A red 'Delete' button is located on the bottom left of the reminder card, and a red arrow points to it.

Email Notification

1. Email notification are sent automatically for upcoming reminders.

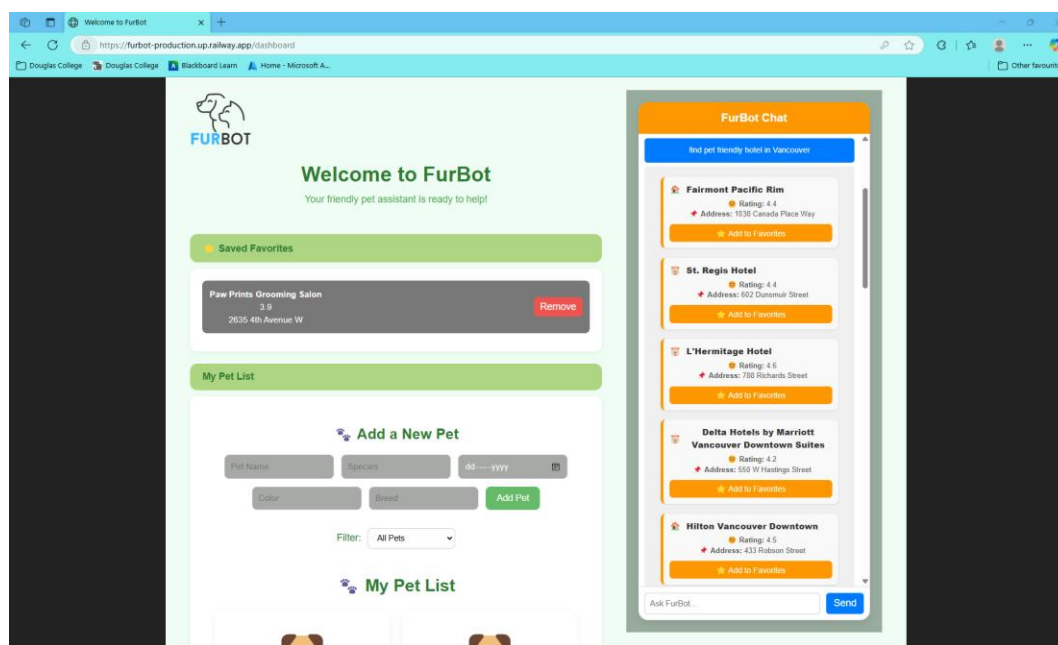
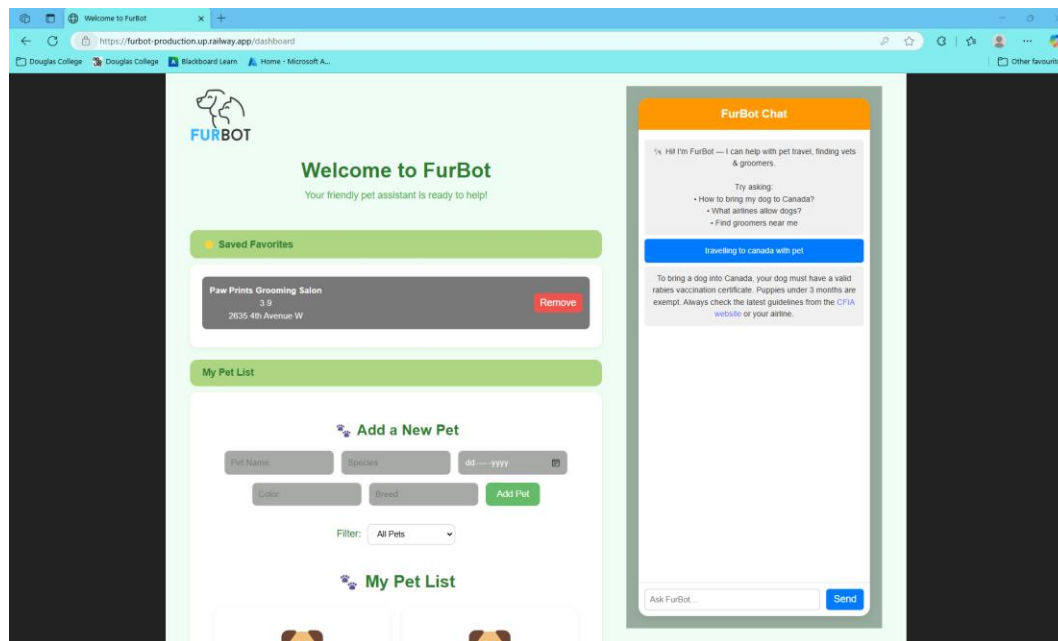


Using the Chatbot

FurBot's chatbot can help you find services and answer pet-related questions.

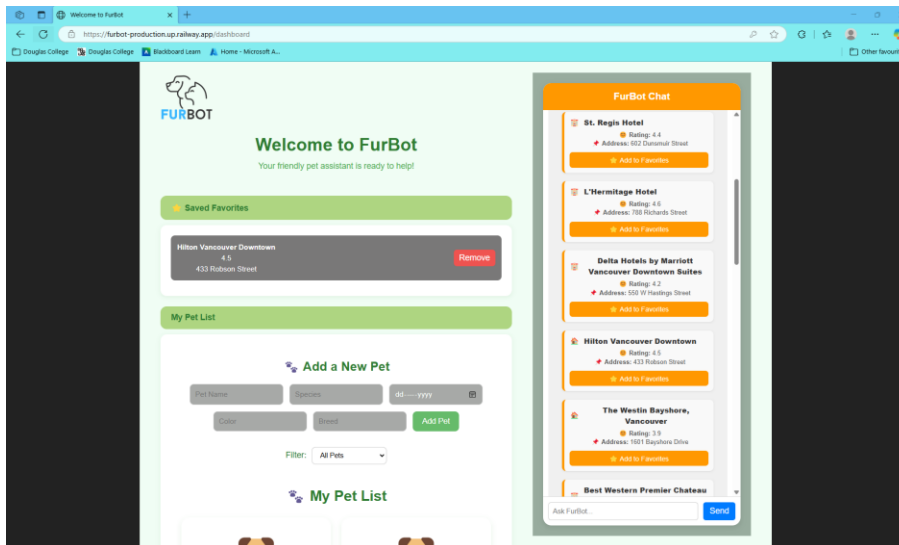
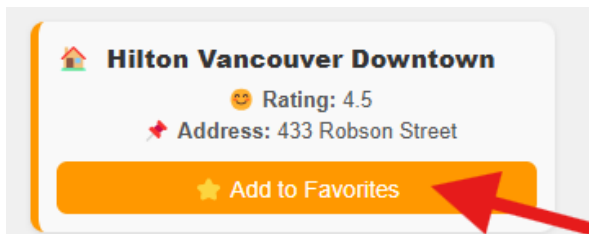
Example commands:

1. "find groomers near Burnaby"
2. "pet friendly hotel in Vancouver"
3. "travelling to Canada with pet"



Adding result to favorites and viewing

1. From the **chatbot results**, locate the business or service you'd like to save.
2. Click **"Add to Favorite"** next to the desired result.
3. The item will be **automatically added** to your personal favorites list.
4. To view saved favorites, go to the **Dashboard** and scroll to the **Favorites** section.



Need Help or Want to Suggest a Feature?

Reach out via contact:

 medialdeac@student.douglascollege.ca

Appendix C: Survey Questionnaire

FurBot User Evaluation Survey Questions

Help us make FurBot better! Tell us about your experience using our pet care assistant.

1.How easy was it to navigate and interact with FurBot?Required to answer. Single choice.

Extremely easy

Somewhat easy

Neutral

Somewhat not easy

Extremely not easy

2.How helpful was FurBot in answering your pet-related questions (e.g., grooming, travel, reminders)?Required to answer. Single choice.

Very helpful

Somewhat helpful

Neither helpful nor unhelpful

Somewhat unhelpful

Very unhelpful

3.Which FurBot feature did you find most useful? (Select one or more)Required to answer. Multiple choice.

Pet Profile Management

Reminder System

Chatbot Assistance

Email Notifications

Other

4.How likely are you to recommend FurBot to other pet owners?Required to answer. Single choice.

Very likely

Somewhat likely

Neither likely nor unlikely

Somewhat unlikely

Very unlikely

5.What suggestions do you have to improve FurBot?Required to answer. Multi Line Text.

Enter your answer